



Manufacturing & Service Operations Management

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Optimal Intraproject Learning

Huan Cao, Nicholas G. Hall, Guohua Wan, Wenhui Zhao

To cite this article:

Huan Cao, Nicholas G. Hall, Guohua Wan, Wenhui Zhao (2024) Optimal Intraproject Learning. Manufacturing & Service Operations Management 26(2):681-700. <https://doi.org/10.1287/msom.2022.0159>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

Copyright © 2023, INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes. For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Optimal Intraproject Learning

Huan Cao,^a Nicholas G. Hall,^b Guohua Wan,^c Wenhui Zhao^{c,*}
^aRobert H. Smith School of Business, University of Maryland, College Park, Maryland 20742; ^bFisher College of Business, The Ohio State University, Columbus, Ohio 43210; ^cAntai College of Economics & Management, Shanghai Jiao Tong University, Shanghai 200030, China

*Corresponding author

Contact: huancao@umd.edu,  <https://orcid.org/0000-0002-7282-6552> (HC); hall.33@osu.edu,  <https://orcid.org/0000-0003-4484-9252> (NGH); ghwan@sjtu.edu.cn,  <https://orcid.org/0000-0002-9298-1023> (GW); zhaowenhui@sjtu.edu.cn,  <https://orcid.org/0000-0002-8037-8016> (WZ)

Received: April 5, 2022

Revised: September 6, 2022; February 21, 2023; July 17, 2023

Accepted: August 9, 2023

Published Online in Articles in Advance: October 10, 2023

<https://doi.org/10.1287/msom.2022.0159>

Copyright: © 2023 INFORMS

Abstract. *Problem definition:* Intraproject learning in project scheduling involves the use of learning among the similar tasks in a project to improve the overall performance of the project schedule. Under intraproject learning, knowledge gained from completing some tasks in a project is used to execute similar later tasks in the same project more efficiently. We provide the first model and solution algorithms to address this intraproject learning problem. *Academic/practical relevance:* Intraproject learning is possible when, for example, the difficulty of the tasks becomes better understood, or the efficiency of the resources used becomes better known. Hence, it is necessary to explore the potential of intraproject learning to further improve the performance of project scheduling. Because learning consumes time, firms may underinvest in intraproject learning if they do not recognize its value. Although the project scheduling literature discusses the potential value of using obtained information from learning within the same project, we formally model and optimize the use of intraproject learning in project scheduling. *Methodology/results:* We model the tradeoff between investing time in learning from completed tasks and achieving reduced durations for subsequent tasks to minimize the total project cost. We show that this problem is intractable. We develop a heuristic that finds near optimal solutions and a strong relaxation that allows some learning from partially completed tasks. Our computational study identifies project characteristics where intraproject learning is most worthwhile. In doing so, it motivates project managers to understand and apply intraproject learning to improve the performance of their projects. A real case is provided by a problem of the Consumer Business Group of Huawei Corporation, for which our model and algorithm provide a greater than 20% improvement in project duration. *Managerial implications:* We find consistent evidence that projects in general can benefit substantially from intraproject learning, and larger projects benefit more. Our computational studies provide the following insights. First, the benefit from learning varies with the features of the project network, and projects with more complex networks possess greater potential benefit from intraproject learning and deserve more attention to learning opportunities; second, noncritical tasks at an earlier project stage should be learned more extensively; and third, tasks that are more similar (or have more similar processes) to later tasks also deserve more investment in learning. Learning should also be invested more in tasks that have more successors, where knowledge gained can be used repetitively.

Funding: This work was supported by the National Natural Science Foundation of China [Grant 71732003 to N. G. Hall and Grants 72131010 and 72232001 to W. Zhao], the Shanghai Subject Chief Scientist Program [Grant 16XD1401700 to G. Wan], and the Program for Professor of Special Appointment (Eastern Scholar) at Shanghai Institutions of Higher Learning [Grant TP2022019 to W. Zhao].

Supplemental Material: The online appendix is available at <https://doi.org/10.1287/msom.2022.0159>.

Keywords: project scheduling • intraproject learning • heuristic algorithm • industry case

1. Introduction

Approximately 30% of the global economy, with an annual value of about \$27 trillion (Hu et al. 2015), is managed as *projects*. Along with controlling scope and cost, the other critical issue in managing projects is meeting a tight deadline (Kerzner 2013). Adding resources such as manpower or equipment may achieve this goal but is typically an expensive option. A cost-effective alternative, especially for unfamiliar projects, is *learning*.

Interproject learning, where knowledge is learned from previously completed projects, is discussed in the literature. However, the focus of our work is *intraproject learning*, that is, learning from earlier tasks within the same project. Both interproject and intraproject can be viewed as classes of *interactivity learning*, that is, learning between activities in projects. When managing a new and unfamiliar type of project, intraproject learning is particularly effective, as

we demonstrate using a consulting project for the Consumer Business Group (CBG) of Huawei Corporation in Shenzhen, China, during 2018–2019.

This paper makes the following contributions to the literature. First, we develop what is apparently the first quantitative model of intraproject learning. Second, we show that the intraproject learning problem is intractable and develop an easily implemented heuristic algorithm that routinely delivers near optimal solutions. Third, we demonstrate the practicality of our model and the substantial value it delivers, using a case study of the supply chain management consulting project for Huawei Corporation. Finally, we test the algorithmic performance, study the sensitivity to misestimation of the input data, and develop managerial insights for implementing intraproject learning in practice. Because the tradeoff between resource consumption and make-span reduction as a consequence of learning is generally not the same within a project as it is between different projects, our work provides valuable insights that are different from those available from interproject learning.

2. Intraproject Learning at Huawei Corporation

The Huawei case is a consulting project for the CBG of Huawei Corporation in Shenzhen, China, during 2018–2019, with the objective of designing and implementing a set of models and tools to help Huawei's CBG manage its smart phone supply chain. Decisions include replenishment quantities, frequencies, and the distribution paths (through air transportation from Shenzhen directly to the retailer warehouses, or through its distribution networks by ground transportation to retailer warehouses) to meet seasonal sales peaks while minimizing the total cost of sales shortages and product markdowns in the supply chain.

The project team consists of professors and graduate students from a university with expertise in operations and supply chain management, together with several staff engineers from Huawei CBG, with the use of historical supply chain data. The outcomes of the project are a set of models and software tools for managing such a supply chain. For example, to forecast the demand for various products at different stages of their life cycles, the team developed several forecasting models based on decision tree regression, namely, random forest, XGBoost, and mixed models of random forest and XGBoost. The computational results on real data sets showed significantly improved forecasting accuracy of the models, compared with the original forecasting models used in the company before (pp. 1077–1078 of Li et al. 2022). Detailed descriptions of the resulting models are available in Li et al. (2022) as follows: a nonlinear programming model for multiperiod

inventory optimization (pp. 1078–1079), a transformed linear programming model based on sample average approximation (p. 1080), three inventory replenishment models based on base stock policy for inventory control (pp. 1080–1081), and the computational results of these models (pp. 1081–1082).

To develop a total solution to demand forecasting and inventory management of highly seasonal products in the supply chain, the consulting team had to accomplish a set of complex tasks on a large amount of available data (such as inventory levels, replenishment quantities and frequencies, product shortages, and markdowns).

In high-level summary, the tasks of the project include the following:

- Data collection and sales pattern analysis during the life cycle of the products;
- Product life cycle identification using real-time sales data;
- Product clustering and demand forecasting with real-time sales data, based on activations;
- Inventory modeling and optimization algorithm development; and
- Testing and simulation with real data of the supply chain's operations.

The project team had little initial knowledge about the special features and characteristics of the tasks in the project. Fortunately, quite a few tasks fall within the same categories and have similar work breakdown structures. Hence, once a task in a category is well understood, the knowledge, tools, and experience can be exploited later for similar tasks in the same category. Consequently, the entire project can be executed more efficiently. In this way, the Huawei project offered a significant opportunity for the project team to implement learning.

During the project execution process, the team (with its leader as a coauthor of this paper) worked with the other authors and exploited the intraproject learning models and algorithms to manage the project to meet the tight due date of the project.

In Section 6, we describe the intraproject learning approach to the Huawei project. It can be seen that our *implementation* of intraproject learning reduces the duration of the project from 454 to 372 days, a saving of 18.1%. Moreover, by our *optimization* model and algorithm for intraproject learning, a further reduction of the project duration to 356 days can be achieved, for a total saving of 21.6%. In this way, the reduced completion time met the company's expectations for expedited project delivery. We emphasize that our work in this paper considers generic projects with a broad variety of characteristics, and this case study is presented as a motivational real-world example and demonstration of how the model and algorithm can be implemented in practice.

3. Literature

Our work extends the classical project scheduling literature, which we now review as a starting point. Brucker et al. (1999) provide a survey of solution methods for the project scheduling problem. For this highly intractable problem, many enumerative and heuristic approaches exist. Brucker et al. (1998) generalize branch-and-bound methods developed for various scheduling problems. Mingozzi et al. (1998) develop improved lower bounds based on preemptive processing of tasks. Möhring et al. (2003) apply Lagrangian relaxation of a time-indexed formulation, along with list scheduling techniques, to obtain strong upper and lower bounds. Kolisch and Hartmann (2006) provide an extensive review of heuristic methods for the project scheduling problem. An updated literature survey is provided by Hartmann and Briskorn (2022), who focus on recent extensions of the project scheduling problem, including preemptive scheduling of tasks, time-dependent resource requirements, setup times, multiple processing modes, and time-cost tradeoffs. They discuss numerous optimal and heuristic solution approaches to the problem. Hall (2016) provides an overview of research and teaching opportunities in project management.

Learning is important in many projects, but especially so for particular types of projects, as discussed in the literature. Bower and Walker (2007) discuss projects that are *staged*, for example, when several buildings within a large construction project are completed and opened for use at different times. A second example is *phased rollout* projects, for example, several different releases of a new software product. In both cases, learning experiences from the early part of the project can help make the later part of the project more efficient. More generally, Rice et al. (2008) emphasize the importance of learning in projects where the ratio of unknowns to knowns is high, for example in research and development or pharmaceutical development projects.

Wang et al. (2021) study the joint effect of truncated learning and controllable processing times for a bicriterion scheduling problem. Van Peteghem and Vanhoucke (2015) study the use of learning curves for task durations in projects and suggest that the timely use of learning effects can generate significant improvements in makespan. However, these works do not assist in solving the intraproject learning problem for two reasons: (1) they do not consider processing times as decision variables that can be changed by investment in learning, and (2) they do not address the tradeoff that arises when time is invested in learning from completed tasks, an activity that diverts resources from progressing the project. Our work models and optimizes project performance using intraproject learning.

For practical procedures for learning within project scheduling, three mechanisms for learning between projects, or interproject learning, are described by

Prencipe and Tell (2001). Julian (2008) discusses how cross-project learning is facilitated by the use of a project management office. Administrative procedures for learning at the end of projects are discussed by Von Zedtwitz (2002), who also comments that many opportunities are lost by neglecting postproject reviews. Schindler and Eppler (2003) discuss a variety of postproject learning methods, for example, process-based methods and documentation-based methods, and recommend implementation approaches. Mangematin et al. (2006) suggest that organizational learning contributes more than standard planning procedures to project success. Sense (2007) recommends the use of supportive situated learning environments within projects as a way to enhance learning. Best practices for documenting lessons learned are described by Simplilearn (2015). Learning in projects occurs between tasks that are similar in subtasks and in resources. In intraproject learning, this similarity is between tasks in the same project, whereas in interproject learning, it is between tasks in a completed project and a future project. The focus of all the previous works is on practical learning procedures and the revelation of the unknown learning benefits within interproject, rather than intraproject, learning. We observe that intraproject learning has not been extensively discussed in the literature. These observations motivate our work. Therefore, our focus is on the modeling and optimization of learning within a single project. We further observe that the potential benefits of intraproject learning are not only unrecognized, but also uncertain, due to the existence of randomness in the project. However, our work is a deterministic one that focuses on the revelation of the unrecognized learning benefits, and we leave the study of uncertain learning benefits to future study.

Intraproject learning requires a tradeoff when learning on a critical task increases the makespan, whereas no such effect occurs between projects in interproject learning. Furthermore, tasks within the same project may have greater learning rates than those in different projects. From a practical perspective, intraproject learning can occur *autonomously* as the project executes. Hill et al. (2021) apply constraint programming approaches to project scheduling with autonomous learning effects. Within the manufacturing scheduling literature, the typical model uses learning by doing (Biskup 2008, Consult Yourself 2015), a type of autonomous learning, where learning effects are realized from repetitive processing of many similar products. However, such volume effects are rarely present within projects, due to their uniqueness (Klastorin 2012, Kerzner 2013).

Although we recognize the usefulness of autonomous learning, our focus is on the additional benefit that can be achieved through investment in *deliberate* learning. The literature of learning within project management supports this perspective. Ayas (1996, p. 131) writes,

“Learning within a project does not happen naturally; it is a complex process that needs to be managed. It requires deliberate attention, commitment, and continuous investment of resources.” Ayas and Zeniuk (2001, p. 64) comment further, “Learning is not a natural outcome of projects and a project-based organization is not necessarily conducive to learning.” The business literature also supports the need for deliberate learning, as discussed by Darter (2014), “Being able to document lessons learned after a project is complete (or even in the middle of it) is one of the biggest responsibilities a project manager has on a project.” Nonetheless, the existing literature of learning within projects is predominantly descriptive in style, in contrast with our work. Moreover, the literature of organizational learning and knowledge transfer (March 1991, Shipton et al. 2002) is not applicable to a project environment where learning predominantly takes place between discrete tasks.

Three works in particular motivate our study of intraproject learning. Kotnour (2000, p. 398) states that “intra-project learning supports the delivery of a successful project” and Kotnour (2000, p. 403) states that “the problem with producing lessons learned at the end of the project is that our biases influence what we will create a lesson learned about.” Turner et al. (2000, p. 17) lament that “Valuable learning experiences take place at the beginning of the project, but are not captured until the postproject review at the end.” Williams (2008) discusses best practices for lessons learned within project management and recommends the documentation of lessons learned within unfinished projects. Despite these statements about the value of intraproject learning, our work is apparently the first to study it from an optimization perspective based on learning between tasks. Additionally, studying the role that intraproject learning plays within the more general topic of interactivity learning motivates our work.

4. General Model

In Section 4.1, we discuss our notation. In Section 4.2, we describe our assumptions and discuss the availability of data used in our model. In Section 4.3, we provide a mathematical model of the problem. Proofs that are not shown in the paper appear in an online appendix.

4.1. Notation

We consider a project defined by a directed acyclic graph $G = (V, E)$, where $V = \{1, \dots, n\}$ is the set of nodes and E is the set of arcs. We use the activities-on-nodes (AoN) convention (Klastorin 2012), where each node in V represents a task. Without loss of generality, we let nodes 1 and n represent dummy tasks at the start and end of the project. Each arc in E defines a

finish-to-start precedence relationship between the tasks. Thus, if $(i, j) \in E$, task i must complete before task j can start. Let $E^T \supseteq E$ denote the extended precedence constraint set that includes all transitive precedence relations. For example, if $E = \{(1, 2), (2, 3)\}$, then $E^T = \{(1, 2), (1, 3), (2, 3)\}$. For each task $i \in V$, we use S_i and C_i to denote decision variables for the start time and completion time of the task, where C_i is dependent on S_i . Hence, if $(i, j) \in E^T$, then $S_j \geq C_i$ in a feasible schedule.

We let t_i denote the nominal processing time of task $i \in V$. The actual processing time of task i , denoted by p_i , depends on the relevant lessons learned from tasks completed before task i starts. The process that generates lessons learned requires time at the end of processing of the task. Until this learning time completes, lessons learned from that task are not usable. The direct decision variables are a start time S_i and an amount of learning time y_i for each task $i = 1, \dots, n$. Then, completion times $C = \{C_1, \dots, C_n\}$ are implied by the values of those variables.

For a given solution $\{S_1, \dots, S_n, y_1, \dots, y_n\}$, let A_i , the *learning set* of task i , denote the set of tasks that complete before task i starts. The lessons learned from set A_i can be applied to task i . Let $\mathbf{A} = \{A_1, \dots, A_n\}$ denote the *learning collection* of a given solution.

A learning set is not exogenously determined by the precedence constraints but depends on how the tasks are scheduled. Thus, it should be endogenized by a solution algorithm. Below, we use this endogeneity to show the NP-hardness of the problem.

Our notation is summarized in Table 1, where we use $\mathbf{X}^t$ to represent the transpose column vector of a row vector $\mathbf{X}$.

4.2. Assumptions and Data

In modeling the problem of optimal intraproject learning, we make the following assumptions:

1. Knowledge from a task is learned by a posttask review process and is only usable on other tasks from the end of that process.

2. Knowledge gained from a posttask review can be applied to all tasks that have not started at the end of that review, but not to other tasks. The justification is that, for a task already in process, task management decisions are typically already determined; hence, the availability of new knowledge during task execution does not reduce the duration of that task.

3. Knowledge gained from a posttask review is a linear function of the time spent, up to a known limit. This assumption can be relaxed to allow for a piecewise linear concave function.

4. The time required by a task i is a nonincreasing, piecewise linear, convex function of the total amount of project knowledge transferred to task i at the time it

Table 1. Notation

Parameters	Description
$G = (V, E)$	Directed graph of precedence constraints representing the project.
$V = \{1, 2, \dots, n\}$	The set of tasks.
$W \subseteq V$	A subset of the tasks.
$E = \{(i, j) \mid i, j \in V\}$	The set of precedence constraints.
$E^T \supseteq E$	The extended set of precedence constraints, including transitive ones.
$f_j(\cdot)$	A function of task j that specifies the amount of reduction in the duration resulting from previous knowledge learned.
$g_j(\cdot)$	A function of task j that specifies the amount of reduction in the duration resulting from autonomous learning.
t_i	Nominal processing time of task i .
p_i	The smallest possible processing time of task i .
$\bar{y}_i$	The maximum learning duration of task i .
q_{ji}	The learning rate between tasks i and j , and thus the proportion of the knowledge gain from learning at task j that is useful for task i .
δ_{ji}	Autonomous learning from task j that is useful for task i .
(Θ^C, Θ^Y)	The coefficient vectors of $\mathbf{C}$ and $\mathbf{Y}$ in the objective function, respectively, normalized into equivalent monetary value.
a_{li}, k_{li}, h_{li}	The processing time reduction rate, knowledge level, and upper bound on the knowledge level, respectively, within the l th interval for task i .
L_i	The number of pieces of the nondecreasing, piecewise linear and concave function representing the task processing time reduction $f_i(\cdot)$.
$\mathbf{d}^t$	The right-hand side column vector, the upper limit of various resources.
$\mathbf{B}^Y$	The coefficient matrix of $\mathbf{Y}$ for resource consumption, where $\mathbf{B}^Y \geq 0$.
Direct decision variables	
y_i	Learning duration of task i .
$\mathbf{S} = \{S_1, S_2, \dots, S_n\}$	Start times of the tasks.
$\mathbf{Y} = \{y_1, y_2, \dots, y_n\}$	Learning durations of the tasks.
$\mathbf{A} = \{A_1, A_2, \dots, A_n\}$	Learning collection, where $A_j = \{i \in V \mid C_i \leq S_j\}$ is the learning set of task j .
Ω	The set of all feasible learning collections.
Indirect decision variables	
$\mathbf{C} = \{C_1, C_2, \dots, C_n\}$	Completion times of the tasks.

starts. The convexity assumption is justified by diminishing marginal returns to further knowledge transfer as the task becomes better learned.

5. Without loss of generality, we assume that the initial knowledge level before the project starts is 0. This assumption can be relaxed as needed.

Most of the data requirements of our model are routinely available within project management practice and research. These include nominal and minimum task time, the maximum time available to review (i.e., learn from) a task, and (where applicable) standard nonrenewable resource constraints. The only two non-standard data requirements have both been the subject of extensive research. The first additional requirement is the learning rate between tasks, which affects the rate of knowledge gain as a result of deliberate learning through posttask performance reviews. Procedures for deliberate learning are documented by Kotnour (2000) and Schindler and Eppler (2003). Zollo and Reuer (2010) develop a model that identifies task similarity as an important factor in learning between tasks. The main attributes of task similarity discussed in the above learning literature are tasks' domain knowledge, tools, and structure, all of which are typically known to

project planners. The second additional requirement is the effectiveness of project knowledge in reducing later task times. The process of intraproject learning is mechanically similar to that for interproject learning, as in Prencipe and Tell (2001) and Julian (2008), with the additional issue being delay during learning that we include in our optimization model. Furthermore, interproject learning can provide additional validation for parameter estimates.

4.3. Learning in General Projects

The problem of optimal intraproject learning to minimize total cost can be written as

$$(P) \quad \min_{\mathbf{A}} \quad G(\mathbf{A}) \quad (1)$$

$$\text{s.t.} \quad \mathbf{A} \in \Omega, \quad (2)$$

where Constraint (2) requires that the learning collection $\mathbf{A}$ is feasible with respect to the precedence constraints defined by E^T . The objective (1) includes the total cost of learning plus a measure of makespan cost. We use a convex penalty based on makespan, as in many project contracts.

For a given learning collection $\mathbf{A} = \{A_1, \dots, A_n\}$, the subproblem is

$$(\bar{P}(\mathbf{A})) \quad G(\mathbf{A}) = \min_{S, Y} H(\mathbf{C}, \mathbf{Y}) \quad (3)$$

$$\text{s.t.} \quad S_i - C_j \geq 0, \quad i \in V, \quad j \in A_i, \quad (4)$$

$$S_i - C_j \leq -\epsilon, \quad i \in V, \quad j \notin A_i, \quad (5)$$

$$C_i - S_i = t_i - f_i \left(\sum_{j \in A_i} q_{ji} y_j \right) - g_i \left(\sum_{j \in A_i} \delta_{ji} \right) + y_i, \quad i \in V, \quad (6)$$

$$0 \leq y_i \leq \bar{y}_i, \quad i \in V, \quad (7)$$

$$\mathbf{B}^Y \mathbf{Y}^t \leq \mathbf{d}^t, \quad (8)$$

where ϵ is a predetermined small positive value, $\mathbf{B}^Y \geq \mathbf{0}$ is the coefficient matrix for consumption of resources for learning, and $\mathbf{d}^t$ is the column vector of the upper limits of various resources. Let $p_i \geq \underline{p}_i = t_i - f_i(\infty) - g_i(\infty) > 0$, where $\underline{p}_i$ is the smallest possible processing time of task i , as maximally reduced by learning. Problem $(\bar{P}(\mathbf{A}))$ contains four parts: the piecewise linear, increasing and convex function $H(\mathbf{C}, \mathbf{Y})$; the learning collection described in Constraint Sets (4) and (5); the piecewise linear, nonincreasing, and convex processing time reduction from learning Constraint Set (6); and the project learning and resource Constraint Sets (7) and (8). In particular, Constraint Set (4) requires that a task i must start after all of its learning sources are completed, and Constraint Set (5) requires that a task that is not a learning source of task i must complete after task i starts. Constraint Sets (4) and (5) together define the learning collection of a given feasible solution. However, following the optimality property developed in Lemma 1, we remove Constraint Set (5) from $(\bar{P}(\mathbf{A}))$ in the rest of the paper. This preserves optimal solutions of (P) .

Lemma 1. Let $(\mathbf{S}^*, \mathbf{Y}^*, \mathbf{A}^*)$ be an optimal solution of Problem (P) . Then, for this optimal solution, Constraint Set (5) is redundant for Problem $(\bar{P}(\mathbf{A}^*))$.

We next discuss the set of learning collections Ω . Because of the precedence constraints, for each pair of tasks (i, j) where $(i, j) \notin E^T$ and $(j, i) \notin E^T$, there are three possible relationships:

1. Task j completes before task i starts, $S_i \geq C_j$: task j is in the learning source set of task i , which we denote by (i, j) .
2. Task i completes before task j starts, $S_j \geq C_i$: task i is in the learning source set of task j , which we denote by (j, i) .
3. Tasks i and j are parallel tasks and neither can use knowledge transferred from the other, which we denote by $(i||j)$ (or equivalently, $(j||i)$).

We use $[i, j]$, for $i < j$, to denote that the relationship of tasks i and j has not yet been determined. Define a set Γ of pairs of such tasks:

$$\Gamma = \{[i, j] \mid i, j \in V, i < j\}. \quad (9)$$

An example of Problem $(\bar{P}(\mathbf{A}))$ appears in the online appendix.

We assume that $H(\mathbf{C}, \mathbf{Y}) = \max\{\Theta_1^C \mathbf{C}^t + \Theta_1^Y \mathbf{Y}^t + \beta_1, \dots, \Theta_L^C \mathbf{C}^t + \Theta_L^Y \mathbf{Y}^t + \beta_L\}$, where $\beta_1, \dots, \beta_L$ and L are constant, and $\Theta_j^C, \Theta_j^Y \geq \mathbf{0}$ for $j = 1, \dots, L$. Thus, $H(\mathbf{C}, \mathbf{Y})$ is an increasing convex piecewise linear function, and $\bar{P}(\mathbf{A})$ can be linearized:

$$(P1(\mathbf{A})) \quad G(\mathbf{A}) = \min_{S, Y, T} T \quad (10)$$

$$\text{s.t. Constraints (4), (6), (7), (8),}$$

$$\Theta_j^C \mathbf{C}^t + \Theta_j^Y \mathbf{Y}^t + \beta_j \leq T, \quad j = 1, \dots, L. \quad (11)$$

Remark 1. An issue related to intraproject learning arises from the learning of noncritical activities to improve the efficiency of later critical activities. The possibility arises that this could eliminate slack from the project and thereby increase the risk to meeting deadlines. Such effects do not occur in our model, because it is deterministic and therefore the duration of each later activity is well defined as a function of previous learning. However, in practice this could be a problem. Where this is likely to be an issue, we could simply adapt our model to control excess learning using Θ_j^Y such that sufficient slack is retained to absorb potential overruns in task duration.

Regarding the solvability of Problem (P) , we have the following result.

Theorem 1. The recognition version of Problem (P) is unary NP-complete.

Remark 2. The proof of Theorem 1 does not rely on the presence of resource constraints. Hence, it illustrates how the transfer of learning between tasks within a project network is sufficient to create intractability, independent of resource bottlenecks. This result holds even if there is no autonomous learning and deliberate learning is cost-free. From Theorem 1, we do not expect to find a polynomial or pseudopolynomial time algorithm for Problem (P) , unless $P = NP$.

Regarding the learning collection, we have the following definition.

Definition 1. A learning collection $\mathbf{A}$ is nested, if $A_j \subseteq A_i$, for $j \in A_i$.

A feasible solution to Problem (P) requires a nested learning collection. However, the number of such collections is exponential in the input size. To circumvent this complexity, we develop upper and lower bounding procedures that endogenize the learning collection so that we do not need to enumerate all possible learning collections.

In Constraint (6), y_j is the learning time at task j , which is in the learning set of task i . Consequently, $\sum_{j \in A_i} q_{ji} y_j$ is the effective learning from all the learning sources of task i , and $f_i(\sum_{j \in A_i} q_{ji} y_j)$ represents the processing time reduction of task i from deliberate learning. Similarly, δ_{ji} represents the learning autonomously gained from performing task j that is useful

for task i , and $g_i(\sum_{j \in A_i} \delta_{ji})$ represents the processing time reduction from autonomous learning. We assume that $g_i(\cdot)$ is a nondecreasing and concave function. As an example of δ_{ji} , let $\delta_{ji} = 1$ for $j \in A_i$, and $\delta_{ji} = 0$ otherwise. Then, $\sum_{j \in A_i} \delta_{ji} = N_i$ is the number of tasks in the learning set of task i . Furthermore, let $g_i(x) = t_i(1 - x^{-\eta_i})$, for a task-specific coefficient η_i . We thus model autonomous learning in the same way as the learning-by-doing literature (Biskup 2008).

The reduction of the processing time of task i from deliberate learning is described by a nondecreasing, piecewise linear, and concave function, that is, $f_i(\sum_{j \in A_i} q_{ji}y_j) = \sum_{l=1}^{L_i} a_{li}k_{li}$, where the number of pieces is denoted by L_i , $a_{1i} > a_{2i} > \dots > a_{L_i i} = 0$ are the processing time reduction rates at different levels of knowledge, $0 \leq k_{li} \leq h_{li} - h_{l-1,i}$ is the knowledge level for task i within the l th interval, and h_{li} denotes an upper bound on the knowledge level within the l th interval, for any $i \in V$, $l = 1, 2, \dots, L_i$, with $h_{0i} = 0$. Then, problem (P1(A)) can be equivalently rewritten as follows:

$$G(\mathbf{A}) = \min_{\mathbf{S}, \mathbf{Y}, T} T$$

s.t. Constraints (4), (7), (8), (11),

$$C_i - S_i = t_i - \sum_{l=1}^{L_i} a_{li}k_{li} - g_i\left(\sum_{j \in A_i} \delta_{ji}\right) + y_i, \quad i \in V, \quad (12)$$

$$\sum_{l=1}^{L_i} k_{li} \leq \sum_{j \in A_i} q_{ji}y_j, \quad i \in V, \quad (13)$$

$$0 \leq k_{li} \leq h_{li} - h_{l-1,i}, \quad i \in V, \quad l = 1, \dots, L_i. \quad (14)$$

Problem (P1(A)) is fundamentally different from the classical project scheduling problem in the sense that, although (P1(A)) is a linear program, its optimal solution depends on the learning collection $\mathbf{A}$, which is itself a function of the decisions $\mathbf{S}$ and $\mathbf{Y}$. This problem extends the model in Möhring et al. (2003) to leverage learning opportunities, but dualizing the resource constraints and using minimum cut computations cannot solve it efficiently. Hence, we develop an iterative procedure to obtain a feasible solution to the overall problem, by solving a series of linear programs, thus also contributing to solution methodologies for project scheduling with contingent constraints.

The concept of delay alternatives, as developed by Christofides et al. (1987) and useful for learning, is extended by Demeulemeester and Herroelen (1992). The following remark discusses the potential use of delay alternatives in intraproject learning.

Remark 3. Because a delay in starting a task may increase the amount of knowledge available to it, and thereby reduce its duration, it is not in general optimal in Problem (P) to start a task immediately after the last of its predecessor tasks completes.

In the remainder of the paper, we denote a feasible solution by $(\mathbf{S}, \mathbf{C}, \mathbf{Y})$, where $\mathbf{S} = (S_1, \dots, S_n)$, $\mathbf{C} = (C_1, \dots, C_n)$ and $\mathbf{Y} = (y_1, \dots, y_n)$. The corresponding learning collection is denoted by $\mathbf{A}$, where A_i is the learning set of task i . We have an optimality condition for a solution to Problem (P).

Lemma 2. For $i \in V \setminus \{1\}$, there exists an optimal solution $(\mathbf{S}^*, \mathbf{C}^*, \mathbf{Y}^*)$ where $S_i^* = C_j^*$ for some task $j \in V$.

From Lemma 2, any task j except task 1 starts at the completion time of some task, which is not necessarily the last completed predecessor of task j because of learning opportunities. We observe that this property is consistent with Remark 3.

5. Heuristic and Lower Bound

Section 5.1 describes a heuristic for finding a feasible solution to problem (P), which provides an upper bound on problem value, and can be used as an easily implemented standalone procedure. Section 5.2 describes lower bounding procedures that validate the quality of the heuristic solutions.

5.1. Heuristic

5.1.1. Procedure AddLearning. We first develop the following procedure that is used as a subroutine in our heuristic.

Procedure AddLearning

0. Let $\mathbf{A}^0 = \mathbf{A} = \mathbf{A}^E$ and $l = 1$.

1. Solve the linear program $P1(\mathbf{A}^{l-1})$, and define $\mathbf{A}^l$ from the optimal solution of $P1(\mathbf{A}^{l-1})$. If $\mathbf{A}^l = \mathbf{A}^{l-1}$, then let $\mathbf{A} = \mathbf{A}^l$, and stop.

2. Set $l = l + 1$, and go to Step 1.

Here, $\mathbf{A}^E = \{A_1^E, A_2^E, \dots, A_n^E\}$ represents the learning collection implied by E^T , the set of precedence constraints including transitivity. For example, for task 4 in Figure EC.2 in the online appendix, $A_4^E = \{1, 2, 3\}$.

Note that $P1(\mathbf{A}^0)$ with $\mathbf{Y} \equiv \mathbf{0}$ is feasible, that is, the solution without any learning is acceptable. This guarantees that $P1(\mathbf{A}^l)$ is feasible for $l > 0$. In Step 1 of Procedure AddLearning, at the l th iteration, we solve the linear program $P1(\mathbf{A}^l)$ to obtain an updated learning collection $\mathbf{A}^{l+1}$, from the sequence of task start and completion times defined by the optimal solution. For example, if $S_i^{l+1} \geq C_j^{l+1}$, that is, task i starts after the completion time of task j in the optimal solution of $P1(\mathbf{A}^l)$, then we add task j into A_i^{l+1} . This process repeats until $\mathbf{A}^{l+1} = \mathbf{A}^l$.

Lemma 3. In AddLearning, we have $A_i^l \subseteq A_i^{l+1}$, for $i \in V$.

Let $\mathcal{L}(n)$ denote the time requirement to solve the linear program $P1(\mathbf{A}^l)$. Then, we have the following result for Procedure AddLearning.

Theorem 2. Procedure *AddLearning* finds a sequence of learning collections with nonincreasing optimal objective values $G(\mathbf{A})$, in $O(n^2\mathcal{L}(n))$ time. The final learning collection is nested, and the final solution found is feasible for the original problem (P).

5.1.2. Procedure FixedLearning. Procedure *AddLearning* generates a feasible solution that provides an upper bound on the optimal objective value. This procedure is a greedy-type heuristic that typically finds a good, but not generally optimal, solution to Problem (P). We now develop a procedure that improves the learning collection $\mathbf{A}$ obtained from Procedure *AddLearning* while keeping the learning time vector $\mathbf{Y}$ fixed. Nevertheless, the completion time of each task still depends on its learning set. Therefore, any adjustment to the learning set of a task may change the completion time of the task, which in turn changes the learning sets of other tasks. As we show in Theorem 1, this effect of the adjustment creates intractability when $\mathbf{Y}$ is also a decision variable. Interestingly, when $\mathbf{Y}$ is given, we show in Theorem 3 that the adjustment is no longer NP-hard, and our procedure finds an optimal learning collection and the smallest completion time of each task in polynomial time. This analysis requires a definition and several preliminary results.

Definition 2. For a nested learning collection $\mathbf{A}$, we call a sequence of $m \geq 2$ tasks, $j_1, j_2, \dots, j_m \in W \subseteq V$, a cycle in set W if $j_1 \in A_{j_2}$, $j_2 \in A_{j_3}$, $\dots$, $j_m \in A_{j_1}$.

Lemma 4. For a nested learning collection $\mathbf{A}$, for any nonempty set $W \subset V$, there must exist a task $i \in W$ such that $A_i \subseteq V \setminus W$.

From Lemma 4, for any nonempty set W that represents the unscheduled tasks, we can find a task $i \in W$, such that all the tasks in i 's learning set are not in W and thus have been scheduled.

We now describe our procedure. Let $G(J, E(J))$ denote the partial network of scheduled tasks, where $J \subset V \setminus \{n\}$ denotes the set of tasks that have already been scheduled, and $E(J)$ denotes the set of precedence relationships between any two tasks in J . Let F denote the partial schedule that specifies the start and completion times of each task in J .

Let $J_c = V \setminus J$ and $J' = \{j \in J_c \mid \nexists (i, j) \in E, i \in J_c\}$. Then, for each $j \in J_c \setminus J'$, there exists at least one $i \in J_c$ where $(i, j) \in E$, else we would have $j \in J'$. Thus, J_c represents the set of tasks remaining to be scheduled, and J' denotes the set of tasks whose predecessor tasks have all been scheduled as completed and are thus available to be scheduled next.

Now, let $l \in J'$ denote the task that is selected to be scheduled next; we discuss how this task is selected. Observe that if there exists $(i, l) \in E$, then $i \in J$, according to the definition of J' . Then, $\max_{(i, l) \in E} \{C_i^F\}$ is the earliest

time when all the predecessors of task l have been scheduled and thus the earliest time at which task l can be scheduled as the next task. Let

$$\tilde{S}_l = \begin{cases} \max_{(i, l) \in E} \{C_i^F\} & \text{if } \{i \in V \mid (i, l) \in E\} \neq \emptyset, \\ 0 & \text{otherwise.} \end{cases} \quad (15)$$

For $l \in J'$, we have $\tilde{S}_l \leq S_l$, because otherwise, the solution $(\mathbf{S}, \mathbf{C}, \mathbf{Y})$ is infeasible.

Although $\tilde{S}_l$ is the earliest possible start time of task l , as mentioned in Remark 3, it may be worthwhile to delay the start of task l to benefit from additional knowledge gained.

To construct a better solution from $(\mathbf{S}, \mathbf{C}, \mathbf{Y})$, we define

$$R_l = \{i \in J \mid C_i^F \geq \tilde{S}_l\}, \quad (16)$$

which represents the set of possible tasks, at the completion of which task l can be scheduled to begin execution, that is, $S_l^F = C_j^F$, for some $j \in R_l$. For example, if $R_l = \{a, b\}$, then we have two options to consider for the start time of task l : $S_l^F = C_a^F$ or $S_l^F = C_b^F$. For any $i \in R_l$, we let

$$T^F(l, i) \equiv C_i^F + t_l - f_l \left(\sum_{\{j \in J \mid C_j^F \leq C_i^F\}} q_{jl} y_j \right) - g_l \left(\sum_{\{j \in J \mid C_j^F \leq C_i^F\}} \delta_{jl} \right) + y_l, \quad (17)$$

which represents the completion time of task l if we start it at the completion time of task i in schedule F . Here, $\sum_{\{j \in J \mid C_j^F \leq C_i^F\}} q_{jl} y_j$ is the total knowledge accumulated from tasks that are scheduled in J before the start of task l , and $g_l(\sum_{\{j \in J \mid C_j^F \leq C_i^F\}} \delta_{jl})$ is the corresponding processing time reduction gained from autonomous learning. Furthermore, we let

$$T^F(l, i) \equiv S_l + t_l - f_l \left(\sum_{\{j \in J \mid C_j^F \leq S_l\}} q_{jl} y_j \right) - g_l \left(\sum_{\{j \in J \mid C_j^F \leq S_l\}} \delta_{jl} \right) + y_l. \quad (18)$$

This represents the completion time of task l if we set $S_l^F = S_l$. Let

$$r(l) = \arg \min_{i \in \{l\} \cup R_l} \{T^F(l, i)\},$$

with ties being broken arbitrarily. Then, task l achieves the smallest completion time if we start it at the completion time of task $r(l)$. Let

$$l^* = \arg \min_{l \in J'} \{T^F(l, r(l))\}.$$

Thus, l^* is the unscheduled task that completes earliest if scheduled next.

Based on the previous discussion, we propose the following procedure that is used as a subroutine in our

heuristic. In the partial network $G(J, E(J))$, we select task $l^* \in J'$ to be scheduled immediately. If $r(l^*) \neq l^*$, we set $S_{l^*}^F = C_{r(l^*)}^F$, that is, task l^* starts at the completion time of task $r(l^*)$. Otherwise, we set $S_{l^*}^F = S_{l^*}$. Next, we set $C_{l^*}^F = T^F(l^*, r(l^*))$. Then, we find an extended partial network $G(J \cup \{j^*\}, E(J \cup \{j^*\}))$. We repeat these steps from $J = \{1\}$ until $J = V$.

Procedure FixedLearning

0. Given $G = (V, E)$ and $(\mathbf{S}, \mathbf{C}, \mathbf{Y})$.
1. Set $S_1^F = C_1^F = 0$, and $J = \{1\}$.
2. If $J = V$, then stop. Otherwise, set $J_c = V \setminus J$ and $J' = \{j \in J_c \mid \nexists (i, j) \in E, i \in J_c\}$.

For $j \in J'$, define $\bar{S}_j$ from (15) and set R_j according to (16).

Let $r(j) = \arg \min_{i \in \{j\} \cup R_j} \{T^F(j, i)\}$, where $T^F(j, i)$ is defined in (17) and (18).

Select task $l^* = \arg \min_{j \in J'} \{T^F(j, r(j))\}$ to schedule next, with ties being broken arbitrarily.

Set the start time of task l^* as $S_{l^*}^F = \begin{cases} C_{r(l^*)}^F, & r(l^*) \neq l^*, \\ S_{l^*}, & r(l^*) = l^*. \end{cases}$

and the completion time of l^* as $C_{l^*}^F = T^F(l^*, r(l^*))$.

Set $J = J \cup \{l^*\}$, and go to Step 2.

The online appendix contains two examples that show how Procedure FixedLearning can be used to reduce cost, and several preliminary results that are required to achieve our main result.

Theorem 3. For any given $\mathbf{Y}$, Procedure FixedLearning finds an optimal solution to problem (P) in $O(n^3)$ time.

From AddLearning, the choice of a given initial feasible learning collection $\mathbf{A}$ and solution of the resulting linear program provide a refined learning collection $\mathbf{A}$ with the corresponding learning time vector. Given this learning time vector, FixedLearning finds an optimal learning collection $\mathbf{A}$.

We now describe a heuristic procedure, DI, as a standalone solution procedure for practical use which iteratively combines Procedures AddLearning and FixedLearning to provide an upper bound on the optimal value of problem (P). The initial learning collection $\mathbf{A}$ is constructed from the set of extended precedence constraints, which is the minimum learning set of each task. Then, using Procedure AddLearning, we derive a locally optimal learning collection $\mathbf{A}$, its corresponding optimal start time vector $\mathbf{S}$, and an optimal learning vector $\mathbf{Y}$. From Theorem 3, using $\mathbf{S}$

and $\mathbf{Y}$ as inputs, we apply Procedure FixedLearning to find the corresponding optimal learning collection $\mathbf{A}$ in $O(n^3)$ time. The refined collection $\mathbf{A}$ is then used as input to AddLearning at later iterations. Finally, we obtain a heuristic solution which provides an upper bound on the optimal value of problem (P). A formal description of our procedure DI follows.

Algorithm (Decreasing Iteration (DI))

0. Initialize $\mathbf{A}$ from E^T .
1. Use Procedure AddLearning to find a locally optimal solution. From this solution, find the corresponding learning time vector $\mathbf{Y}$ and learning collection $\mathbf{A}$.
2. Use Procedure FixedLearning to find an improved learning collection $\mathbf{A}^*$ for the current learning time vector $\mathbf{Y}$ from Step 1. If this new $\mathbf{A}^*$ is unchanged from Step 1, that is, $\mathbf{A}^* = \mathbf{A}$, or if the total reduction in the objective value over five consecutive iterations is small enough to meet a prespecified tolerance, then stop. Otherwise, let $\mathbf{A} = \mathbf{A}^*$ and go to Step 1.

5.2. Lower Bounds

We design procedures to find lower bounds that validate the quality of solutions found by Algorithm DI. To construct lower bounds, we relax the learning set requirement to find an objective value that is less than or equal to the optimal one. To achieve this, we impose two requirements. First, all the learning sources of a task i in Problem (P) are still learning sources of task i in the relaxed problem. Second, some other tasks which are not learning sources of task i in Problem (P) can still contribute partial knowledge to task i in the relaxed problem.

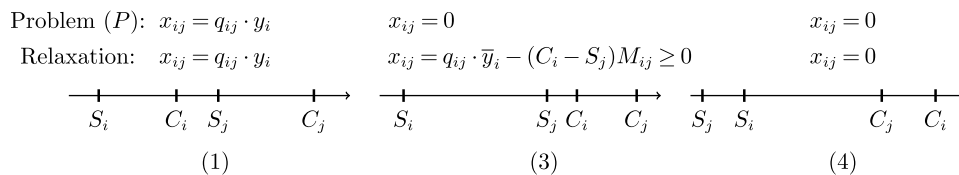
To achieve this goal, we consider tasks i and j where neither (i, j) nor (j, i) is in E^T , that is, neither task is initially a learning source of the other. Let $x_{ij} \geq 0$ denote the amount of knowledge that task j can gain through active learning from task i . We relax the knowledge in function $f_i(\cdot)$, using

$$x_{ij} = \min\{q_{ij}y_i, \max\{0, q_{ij}\bar{y}_i - (C_i - S_j)M_{ij}\}\}, \quad (19)$$

where $M_{ij} > 0$ is a constant with value that is discussed later. Then, the value of x_{ij} depends on $C_i - S_j$ under several cases illustrated in Figure 1:

1. If $S_j \geq C_i$, we have $x_{ij} = q_{ij} \cdot y_i$. That is, task j can use all the knowledge created by task i , i.e., task i is a

Figure 1. Original Problem and Relaxation



full learning source of j . This is also true in Problem (P), and hence the relaxation is exact.

2. If $S_j < C_i$ and $C_i - S_j$ is sufficiently small, we still have $x_{ij} = q_{ij} \cdot y_i$. Then, task j can still use all the knowledge created by task i , that is, i is a full learning source of j . However, in Problem (P), because task i finishes after task j starts, task j cannot use any knowledge created by task i .

3. If $S_j < C_i$ and $C_i - S_j$ is in a middle range, we have $0 \leq x_{ij} = q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} < q_{ij} \cdot y_i$. That is, task j can use partial knowledge created by task i , that is, i is a partial learning source of j . However, in Problem (P), task j cannot use any knowledge created by task i .

4. If $S_j < C_i$ and $C_i - S_j$ is sufficiently large, we have $x_{ij} = 0$. That is, task j cannot use any knowledge created by task i , that is, i is not a learning source of j . In Problem (P), task i is similarly not a learning source of task j , hence the relaxation is exact.

Observe that, in Cases 2 and 3, the relaxation allows transfer of more knowledge for task j . That is, task j can also use a portion of the knowledge gained by task i that depends on $C_i - S_j$ and M_{ij} if $C_i - S_j > 0$ is not too large.

To model Constraint (19), we let $x_{ij}^+ = x_{ij} \geq 0$ denote the knowledge that task j can use from task i , and introduce dummy variables $x_{ij}^- \geq 0$. Then,

$$x_{ij}^+ - x_{ij}^- \leq q_{ij}y_i, \quad (20)$$

$$x_{ij}^+ - x_{ij}^- \leq q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij}, \quad (21)$$

$$x_{ij}^+ \cdot x_{ij}^- = 0, \quad (22)$$

$$x_{ij}^+, x_{ij}^- \geq 0. \quad (23)$$

Whenever $q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} > 0$, we have $x_{ij}^+ > 0$ and $x_{ij}^- = 0$ so that task j gains knowledge from task i , which may reduce the objective value. However, if $q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} \leq 0$, then Constraint (21) requires that $x_{ij}^- \geq 0$ and $x_{ij}^+ = 0$, and no knowledge transfers from task i to j .

Similarly for autonomous learning, we relax the function $g(\cdot)$, which gives

$$u_{ij} = \min\{\delta_{ij}, \max\{0, \delta_{ij} - (C_i - S_j)W_{ij}\}\}, \quad (24)$$

where $W_{ij} > 0$ is a constant with value that is discussed later. Then, the value of u_{ij} depends on $C_i - S_j$, under the following three cases:

1. If $S_j \geq C_i$, we have $u_{ij} = \delta_{ij}$.
2. If $S_j < C_i$ and $C_i - S_j$ is sufficiently small, we have $0 \leq u_{ij} = \delta_{ij} - (C_i - S_j)W_{ij} < \delta_{ij}$.
3. If $S_j < C_i$ and $C_i - S_j$ is sufficiently large, we have $u_{ij} = 0$.

To model Constraint (24), we let $u_{ij}^+ = u_{ij} \geq 0$ denote the autonomous knowledge that task j can use from task i , and introduce dummy variables $u_{ij}^- \geq 0$. Then,

$$u_{ij}^+ - u_{ij}^- \leq \delta_{ij}, \quad (25)$$

$$u_{ij}^+ - u_{ij}^- \leq \delta_{ij} - (C_i - S_j)W_{ij}, \quad (26)$$

$$u_{ij}^+ \cdot u_{ij}^- = 0, \quad (27)$$

$$u_{ij}^+, u_{ij}^- \geq 0. \quad (28)$$

Whenever $\delta_{ij} - (C_i - S_j)W_{ij} > 0$, we have $u_{ij}^+ > 0$ and $u_{ij}^- = 0$. However, if $\delta_{ij} - (C_i - S_j)W_{ij} \leq 0$, then Constraint (26) requires that $u_{ij}^- \geq 0$ and $u_{ij}^+ = 0$.

Let $\mathbf{X}^+ = \{x_{ij}^+\}$, $\mathbf{X}^- = \{x_{ij}^-\}$, $\mathbf{U}^+ = \{u_{ij}^+\}$ and $\mathbf{U}^- = \{u_{ij}^-\}$. We model the relaxed problem as follows:

$$(PR) \quad \min_{s, \mathbf{Y}, T, \mathbf{X}^+, \mathbf{X}^-, \mathbf{U}^+, \mathbf{U}^-} T \quad (29)$$

s.t. Constraints (7), (8), (11),

Constraints (20)–(23), (25)–(28),

$$i, j \in V, i \neq j, (i, j) \notin E^T, (j, i) \notin E^T,$$

$$x_{ij}^+ \cdot x_{ji}^+ = 0, \quad i, j \in V, i \neq j, (i, j) \notin E^T, (j, i) \notin E^T, \quad (30)$$

$$u_{ij}^+ \cdot u_{ji}^+ = 0, \quad i, j \in V, i \neq j, (i, j) \notin E^T, (j, i) \notin E^T, \quad (31)$$

$$C_i - S_i = t_i - f_i \left(\sum_{j=1, j \neq i}^n x_{ji}^+ \right) - g_i \left(\sum_{j=1, j \neq i}^n u_{ji}^+ \right) + y_i, \quad i \in V, \quad (32)$$

$$S_j - C_i \geq 0, \quad (i, j) \in E, \quad (33)$$

$$x_{ij}^+ - q_{ij}y_i \leq 0, \quad (i, j) \in E^T, \quad (34)$$

$$x_{ji}^+ = 0, \quad (i, j) \in E^T, \quad (35)$$

$$u_{ij}^+ - \delta_{ij} \leq 0, \quad (i, j) \in E^T, \quad (36)$$

$$u_{ji}^+ = 0, \quad (i, j) \in E^T. \quad (37)$$

Objective Function (29) includes the vectors $\mathbf{X}^+$, $\mathbf{X}^-$, $\mathbf{U}^+$ and $\mathbf{U}^-$ as decision variables. Constraints (20)–(23) and (25)–(28) implement the relaxation of knowledge transfer. Constraints (6) are replaced by Constraints (32). To strengthen the relaxation, Constraints (30) and (31) require that no two tasks, even without a precedence relationship, can both learn from each other, that is, if task i gains knowledge from task j , then task j cannot gain knowledge from task i . This restriction also applies to Problem (P). Constraints (33) model the precedence constraint requirements. Constraints (34)–(37) guarantee that if task i is a predecessor of task j , then task j can gain knowledge from task i , whereas task i cannot gain knowledge from task j .

We now linearize the four sets of complementarity constraints, (22), (27), (30), and (31), so that Problem (PR) becomes a linear program. Recall that $\bar{y}_i$ is the upper limit on the learning time of task i ; hence, $x_{ij}^+ \leq q_{ij}\bar{y}_i$. Let $\bar{x}_{ij}^-$ be an upper bound on the dummy variable x_{ij}^- , that is,

$$0 \leq x_{ij}^- \leq \bar{x}_{ij}^-, \quad \forall i, j \in V, i \neq j.$$

Note that $(x_{ij}^+ - q_{ij}\bar{y}_i)(x_{ij}^- - \bar{x}_{ij}^-) \geq 0$. Because $x_{ij}^+ \cdot x_{ij}^- = 0$ from (22), we have

$$\frac{x_{ij}^+}{q_{ij}\bar{y}_i} + \frac{x_{ij}^-}{\bar{x}_{ij}^-} \leq 1, \quad \forall i, j \in V, i \neq j. \quad (38)$$

Note that $u_{ij}^+ \leq \delta_{ij}$, and let $\bar{u}_{ij}^-$ be an upper bound on the dummy variable u_{ij}^- . Similarly, we linearize Constraints (27), (30), and (31) as follows:

$$\frac{u_{ij}^+}{\delta_{ij}} + \frac{u_{ij}^-}{\bar{u}_{ij}^-} \leq 1, \quad \forall i, j \in V, i \neq j, \quad (39)$$

$$\frac{x_{ij}^+}{q_{ij}\bar{y}_i} + \frac{x_{ji}^+}{q_{ji}\bar{y}_j} \leq 1, \quad \forall i, j \in V, i \neq j, \quad (40)$$

$$\frac{u_{ij}^+}{\delta_{ij}} + \frac{u_{ji}^+}{\delta_{ji}} \leq 1, \quad \forall i, j \in V, i \neq j. \quad (41)$$

Replacing Constraints (22), (27), (30), and (31) with (38)–(41), respectively, yields a new relaxed problem:

$$\begin{aligned} (PR1) \quad & \min_{\mathbf{s}, \mathbf{y}, T, \mathbf{x}^+, \mathbf{x}^-, \mathbf{u}^+, \mathbf{u}^-} T \\ \text{s.t.} \quad & \text{Constraints (7), (8), (11), (32)–(37),} \\ & \text{Constraints (20), (21), (23), (25), (26), (28),} \\ & \text{(38)–(41), } i, j \in V, i \neq j, (i, j) \notin E^T, (j, i) \notin E^T. \end{aligned}$$

We next find useful upper bounds on $\bar{x}_{ij}^-$ and $\bar{u}_{ij}^-$. We have the following result.

Lemma 5. For any $i, j \in V$, suppose we can find an upper bound $\bar{C}_i$ on C_i^* and a lower bound $\underline{S}_j$ on S_j^* . Hence, we restrict our attention to solutions with $C_i \leq \bar{C}_i$ and $S_j \geq \underline{S}_j$. Let

$$\bar{x}_{ij}^- \equiv \max\{0, -(q_{ij} \cdot \bar{y}_i - (\bar{C}_i - \underline{S}_j)M_{ij})\}, \quad (42)$$

$$\bar{u}_{ij}^- \equiv \max\{0, -(\delta_{ij} - (\bar{C}_i - \underline{S}_j)W_{ij})\}. \quad (43)$$

Then $\bar{x}_{ij}^-$ is a valid upper bound on x_{ij}^- and $\bar{u}_{ij}^-$ is a valid upper bound on u_{ij}^- .

We observe that M_{ij} and W_{ij} are constants to be chosen. For notational convenience, let $\mathbf{M} = (M_{ij})$ and $\mathbf{W} = (W_{ij})$, two $n \times n$ matrices. We have the following result about the values of $\mathbf{M}$, $\mathbf{W}$ and the corresponding optimal value of the objective function $Obj^*(PR(\mathbf{M}, \mathbf{W}))$.

Theorem 4. The optimal value of Problem (PR) is nondecreasing in $\mathbf{M}$ and $\mathbf{W}$, and $Obj^*(PR(\mathbf{M}, \mathbf{W})) \rightarrow Obj^*(P)$ as $\mathbf{M} \rightarrow +\infty$ and $\mathbf{W} \rightarrow +\infty$.

Theorem 4 shows that larger values of M_{ij} and W_{ij} , at least up to a point to be identified later, are helpful in strengthening the relaxation. However, in problem (PR1), larger values of $\mathbf{M}$ and $\mathbf{W}$ result in larger values of $\bar{x}_{ij}^-$ and $\bar{u}_{ij}^-$, respectively, which makes Constraints (38) and (39) weaker and thus makes the lower bound from the relaxation looser. Hence, we need values of $\mathbf{M}$ and $\mathbf{W}$ that result in the strongest lower bound from problem (PR1), as identified in the following result.

Theorem 5. The values of M_{ij} and W_{ij} , for any $i, j \in V, i \neq j, (i, j) \notin E^T$ and $(j, i) \notin E^T$, that provide the strongest relaxation (PR1) are $M_{ij} = \frac{q_{ij}\bar{y}_i}{\bar{C}_i - \underline{S}_j}$ and $W_{ij} = \frac{\delta_{ij}}{\bar{C}_i - \underline{S}_j}$.

If we set M_{ij} and W_{ij} as in Theorem 5, we have $\bar{x}_{ij}^- = 0$ and $\bar{u}_{ij}^- = 0$. Hence, we rewrite (PR1) as another linear program (PR2), by setting all $x_{ij}^- = \bar{x}_{ij}^- = 0$, $u_{ij}^- = \bar{u}_{ij}^- = 0$, and replacing Constraints (21) and (38), and (26) and (39), with

$$x_{ij}^+ \leq q_{ij} \cdot \bar{y}_i \left(1 - \frac{C_i - S_j}{\bar{C}_i - \underline{S}_j}\right), \quad (44)$$

$$u_{ij}^+ \leq \delta_{ij} \left(1 - \frac{C_i - S_j}{\bar{C}_i - \underline{S}_j}\right), \quad (45)$$

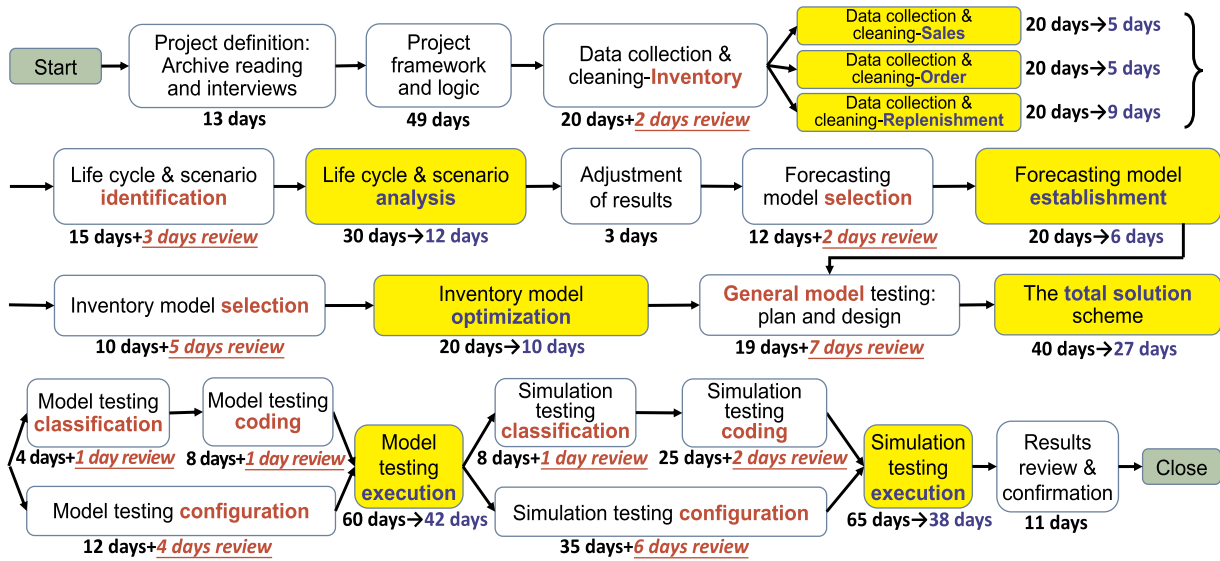
respectively. This relaxation is as follows:

$$\begin{aligned} (PR2) \quad & \min_{\mathbf{s}, \mathbf{y}, T, \mathbf{x}^+, \mathbf{x}^-, \mathbf{u}^+, \mathbf{u}^-} T \\ \text{s.t.} \quad & \text{Constraints (7), (8), (11), (32)–(37),} \\ & \text{Constraints (20), (23), (25), (28), (40), (41), (44), and (45),} \\ & i, j \in V, i \neq j, (i, j) \notin E^T, (j, i) \notin E^T, \\ & C_i \leq \bar{C}_i, \quad i \in V, \quad (46) \\ & S_i \geq \underline{S}_i, \quad i \in V. \quad (47) \end{aligned}$$

To summarize, if we know the bounds $\bar{C}_i$ and $\underline{S}_i$ for each task i , we solve problem (PR2) to find a lower bound on the optimal value of Problem (P). These bounds are updated within the branch-and-bound algorithm as more information becomes available, as discussed in the online appendix.

6. Solution of Huawei Project Case Study

We refer to the Huawei project briefly described in Section 1. From a high-level perspective, the project consists of 24 main tasks, as shown in Figure 2, where the several highlighted boxes, except the ‘start’ and ‘end’ boxes, represent the tasks that benefit from learning (i.e., represent tasks whose processing times are reduced). Where a task time is shown in the format “ x days $\rightarrow$ z days,” the nominal processing time is x days and the reduced processing time after learning from earlier tasks is z ($< x$) days. The learning time is shown in the format “ y days review.” The total estimated processing time is 454 days, or about 15 months. One reason for this long duration is that, although the team workers were sufficiently available, the bottleneck in the process was to achieve a detailed understanding and accurate modeling of the company’s operations. This was initially achieved by in-depth discussions between the senior staff and the company staff, and the obtained information was then shared with the team workers working on the project. For this project, Huawei requested an expedited completion time of 12 months, or earlier if possible, to allow for full implementation before the annual sales peak. To meet this tight project deadline, it was decided that the senior staff had to be devoted to

Figure 2. (Color online) Huawei Project Network

learning from early tasks (together with the company staff) in the project and then sharing the learned knowledge with the team workers. This was accomplished by using, for each main task,

- An estimate of learning rates among tasks and identification of potential learning source tasks,
- A detailed review and training session, that is, intraproject learning, run by the senior staff and shared with the team workers, and
- A large-scale implementation by team workers and Huawei information technology (IT) staff.

Thus, between several pairs of tasks, there is a potential period of review to generate learning and improve the efficiency of the later task(s), in some cases significantly. The tradeoff in this process is that the review and training session delays the following tasks of the project. However, the net benefit in project performance, if the review and training are done appropriately, is still considerable. Our *implemented* application of intraproject learning reduces the duration of the project from 454 to 372 days, a saving of 18.1%. Moreover, the *optimization* of intraproject learning, using the model and algorithm described later, further reduces the project duration to 354 days, for a total saving of 22% and a further 4.8% saving from the performed duration of 372 days. Thus, the optimized completion time met the company's 12-month expectation for project delivery.

We describe how the model in Section 4 and solution procedures in Section 5 are used to solve the project problem at Huawei Corporation. Table 2 summarizes the data in the project, where we use y_j to denote the learning time on task j , which is a decision variable in the model. As a result of the network structure of the project and the elimination criteria (Lemmas EC.7 and EC.8 in the online appendix), the learning collection A

is fixed and the learning sets A_j consist of all the tasks earlier than j in the network. For simplicity, all the q_{ij} values not shown in the fifth column of Table 2 are zero (and the corresponding tasks are not shown in A_j). For example, according to the definition of a learning set, we know $A_4 = \{1, 2, 3\}$. However, only task 3 appears in A_4 because q_{14} and q_{24} are zero.

For a task j that learns from task i , following Levy (1965) and Wang et al. (2010), we model the actual processing time of task j as

$$p_j = t_j \left[\alpha_j e^{-b \sum_{i \in A_j} q_{ij} y_i} + (1 - \alpha_j) \right] = t_j [\alpha_j e^{-b K_j} + (1 - \alpha_j)], \quad (48)$$

where $K_j = \sum_{i \in A_j} q_{ij} y_i$ represents the knowledge task j can use. This is a special case of the general model in Section 4.3 and the corresponding processing time reduction function of task j , $f_j(K_j)$, equals $t_j \alpha_j (1 - e^{-b K_j})$. Here, $0 \leq 1 - \alpha_j \leq 1$ indicates the limit of the learning effect, $p_j = t_j(1 - \alpha_j)$, and thus we have $\alpha_j = 1 - p_j/t_j$. Also, $0 \leq q_{ij} \leq 1$ indicates the learning rate between tasks i and j , which can be determined by the domain knowledge of the tasks, for example their breakdown structures, types of work, and execution methods used.

In the Huawei project, tasks 10 and 11 are concerned with forecasting. Task 10 compares and selects off-the-shelf forecasting models, and task 11 makes necessary customizations to the selected models to define the forecasting models that are finally used. The similarity of the two tasks makes the application of learning between them easier. The most difficult and time-consuming part of task 11 is to handle those SKUs that do not show clear features. Thus, a review at the end of task 10 to compare and contrast various categories of SKU is important to reduce the duration of task 11

Table 2. Data and Optimal Solution for Huawei Project

Task	Description	A_j	t_j	α_j	q_{ij}	Actual LT	Reduced PT	Optimal LT	y_j	Optimal PT
1	Project definition		13							
2	Project framework		49							
3	Inventory data collection and cleaning		20			2		6.8		
4	Sales data collection and cleaning	3	20	0.85	0.5		5			3
5	Orders data collection and cleaning	3	20	0.85	0.5		5			3
6	Replenishment data collection and cleaning	3	20	0.85	0.35		9			3
7	Life cycle identification		15			3		4.7		
8	Life cycle analysis	7	30	0.7	0.5		12			9
9	Adjustment		3							
10	Forecasting model selection		12			2		3.2		
11	Forecasting model establishment	10	20	0.85	0.75		6			3
12	Inventory model selection		10			5		2.2		
13	Inventory model optimization	12	20	0.7	0.25		10			11.6
14	Model integration		19			7		9.5		
15	Global solution development	14	40	0.5	0.25		27			20
16	Model testing classification		4			1		0		
17	Model testing configuration		12			4		4		
18	Model testing coding		8			1		4		
19	Model testing execution	{16,17,18}	60	0.35	{0.05,0.5,0.1}		42			39
20	Simulation testing classification		8			1		0		
21	Simulation testing configuration		35			6		6.2		
22	Simulation testing coding		25			2		8.2		
23	Simulation testing execution	{20,21,22}	65	0.5	{0.05,0.25,0.1}		38			32.5
24	Results confirmation		11							

Note. LT, learning time; PT, processing time; t_j , PT, and LT are all in days.

effectively. In fact, the learning rate between the two tasks is high, estimated at 0.75 in our model.

Observe that b represents the knowledge response of the project team, which is equivalent to a_{ij} in the general model. When b is close to zero, p_j is close to t_j hence the knowledge response of the team is weak, and when b is very large, p_j is close to $t_j(1 - \alpha_j) = p_j$ hence the knowledge response of the project team is strong. The knowledge response can be estimated from historical data (Levy 1965, Adler and Clark 1991, Lapré et al. 2000). For the company case, it can be estimated using our implemented application data. As shown in Table 2, the heuristic solution provides the learning data which includes nine (learning time, reduced processing time) pairs (y_i, p_i) , and b is estimated from this data using ordinary least squares. Specifically, let $p_j^\dagger = \ln\left(\frac{t_j \alpha_j}{p_j - t_j(1 - \alpha_j)}\right)$, where the superscript $\dagger$ denotes that it is the linearized processing time term. Then, from (48), we have

$$p_j^\dagger = b \sum_{i \in A_j} q_{ij} y_i + \eta_j, \quad (49)$$

Table 3. Estimation Result for b

	Estimate	Standard error
Knowledge response (b)	1.1***	(0.1464)
R^2	0.8758	
Observations	9	

Note. Standard errors in parentheses.

* $p < 0.10$; ** $p < 0.05$; *** $p < 0.01$.

where η_j represents the error term. The regression results of (49) are provided in Table 3. The value of $R^2 = 0.8758$ indicates that the learning performance of the project team is consistent. Therefore, we believe that $\hat{b} = 1.1$ is a reasonable estimate of the knowledge response of the project team of the company case. Also, because the standard error is 0.1464, we use a standard deviation of 0.15 in our parameter misestimation analysis.

Because e^{-bK_j} is a convex curve, we approximate it using three piecewise linear segments, that is, $L_j = 3$. For a more complex project, the use of additional linear segments might provide greater accuracy. When $bK_j = 1.61$, we have $e^{-1.61} = 0.2$. The gradient of the tangent is -0.2 . The line passing through $(1.61, 0.2)$ with gradient -0.2 is $z = 0.522 - 0.2bK_j$. When K_j is small, we use the line $z = 1 - bK_j$ to approximate the curve e^{-bK_j} , and when bK_j is midrange, we use $z = 0.522 - 0.2bK_j$. The two lines $z = 1 - bK_j$ and $z = 0.522 - 0.2bK_j$ intersect at $bK_j = 0.5975 \approx 0.6$, and the two lines $z = 0.522 - 0.2bK_j$ and $z = 0$ intersect at $bK_j = 2.61 \approx 2.6$. Thus, we approximate $t_j[\alpha_j e^{-bK_j} + (1 - \alpha_j)] = t_j[1 - \alpha_j(1 - e^{-bK_j})]$ using

$$\begin{cases} t_j(1 - \alpha_j b K_j), & 0 \leq K_j < \frac{0.6}{b}, \\ t_j(1 - 0.48\alpha_j - 0.2\alpha_j b K_j), & \frac{0.6}{b} \leq K_j < \frac{2.6}{b}, \\ t_j(1 - \alpha_j), & K_j \geq \frac{2.6}{b}. \end{cases} \quad (50a)$$

$$(50b)$$

$$(50c)$$

For any task i in A_j , once the learning time of task i is larger than $\frac{2.6}{bq_{ij}}$ days, additional learning does not help.

Therefore, we use $\bar{y}_i = \max \left\{ \frac{2.6}{bq_{ij}} \mid q_{ij} > 0 \right\}$ as the maximum learning time of task i . Observe that once we know α_j , q_{ij} and b , the lines and the value of $\bar{y}_i$ are completely determined. An example of this data modeling and the whole formulation of the case appear in the online appendix.

After substituting the estimated value of b into the model, our algorithm can be applied, and the optimal solution is presented in the last two columns of Table 2. Compared with the actual learning times performed by the team, most of the optimized learning times are slightly larger, with the exception of those for tasks 12, 16, and 20. This illustrates that, although most of the time project managers underinvest in intraproject learning, too much learning can also backfire on the current project (e.g., when learning is inefficient but the learning time effectively increases the project duration, especially when it happens along the critical path). Our work identifies and evaluates a potential benefit that has not been fully appreciated by project companies. More general managerial insights about intraproject learning are described in Section 7.3.

Although Zollo and Reuer (2010) characterize the main attributes of learning rate between tasks as the similarity of their domain knowledge, tools, and structure, which are not hard to quantify, we still perform a parameter misestimation study for the case project. Specifically, we allow all the q_{ij} and b values for each application of learning to have estimation errors; the percentage estimation errors are assumed to be independent and identically distributed (i.i.d.) and follow a $N(0, 0.15^2)$ distribution. From this analysis, the additional 4.8% improvement is found to be significant at the 99% confidence level. We conclude that a formal model of intraproject learning can provide significant value to a real-world project. A comprehensive parameter misestimation study for a general project can be found in Section 7.2.

Moreover, for projects in general, we expect the share of makespan improvement from optimization of intraproject learning to be larger than in the Huawei project for two reasons. First, the Huawei project has a small number of tasks. Consequently, learning decisions are relatively easy to estimate heuristically. Second, the Huawei project has a relatively simple precedence structure that is close to serial. More complex precedence structures that typically occur in projects would make it difficult to find good quality heuristic learning solutions. Such a case in another currently ongoing project in Huawei's Global Technical Services department led by one of the authors shows the potential of our model and algorithm in intraproject learning. Our computational results for randomly generated large and complex projects in the last paragraph of Section 7.3.1 validate these expectations.

7. Computational Study

In Section 7.1, we provide a robustness check on the computational performance of our algorithms for projects in general. Section 7.2 contains a sensitivity analysis to parameter misestimation. In Section 7.3, we conduct controlled experiments to generate useful managerial insights.

In Sections 7.1–7.3, for concreteness, we consider a practical case of Objective (1), which requires minimizing $\alpha C_n + \sum_{i=1}^n c_i y_i$. We specify Constraint Set (8) as $\sum_{i=1}^n m_i y_i \leq R$, where $\alpha = 0.9$ and R is the amount of resource available for learning. Furthermore, for simplicity, we assume nonexistence of autonomous learning, and specify the processing time reduction function $f(\cdot)$ as $f_i(\sum_{j \in A_i} q_{ji} y_j) = \sum_{l=1}^{L_i} a_{li} k_{li}$, where $a_{1i} > a_{2i} > \dots > a_{L_i i} = 0$.

Test instances are randomly generated as follows. For an instance with n tasks, we set the resource quantity as $R = 20n$, and the resource consumption rate for each task, m_i , as generated from a discrete uniform distribution $U[0.1, 1]$ with step size 0.01. For each task i , we set the learning cost coefficient, c_i , as generated from a discrete uniform distribution $U[0.02, 0.1]$ with step size 0.001; the minimum task processing time, p_i , as generated from a discrete uniform distribution $U[0.5t_i, 0.8t_i]$ with step size 1, where t_i is the nominal task processing time; the maximum learning time, $\bar{y}_i$, as generated from the discrete uniform distribution $U[t_i/10, t_i/5]$ with step size 1; and elements of the learning rate matrix, $\{q_{ji}\}$, as generated from the discrete uniform distribution $U[0, 1]$ with step size 0.1. For the processing time function, we set the number of linear pieces as $L_i = 3$, and set the knowledge response a_{1i} as generated from the discrete uniform distribution $U[0.1, 0.5]$ with step size 0.005; and a_{2i} as generated from the discrete uniform distribution $U[a_{1i}/4, a_{1i}/2]$ with step size 0.005 and $a_{3i} = 0$. We set h_{1i} as generated from the discrete uniform distribution $U\left[0.2 \frac{t_i - p_i}{a_{1i}}, 0.7 \frac{t_i - p_i}{a_{1i}}\right]$

with step size 1 and $h_{2i} = h_{1i} + \frac{(t_i - p_i) - a_{1i}h_{1i}}{a_{2i}}$. Finally, we set h_{3i} as a large number. All procedures are coded in Microsoft Visual C++ Studio 2017 with IBM ILOG CPLEX Optimization Studio 12.7.1 and run on a 3.60-GHz Intel Core CPU computer with 32 GB of memory.

7.1. Robustness Check of the Algorithm Performance

We evaluate the computational performance of our heuristic described in Section 5 and our branch and bound algorithm described in the online appendix for Problem (P) in general. For each parameter combination specified later, we generate 30 random networks using RanGen1 (Demeulemeester et al. 2003). RanGen1 generates networks with a given number n of non-dummy tasks with uniformly distributed task times t_i , for $i = 1, 2, \dots, n$, and prespecified density or order

Table 4. Sensitivity to Number of Tasks, Order Strength, a_{1i} , c , m , and q_{ji}

Parameter	Cost_Red. (%)	IG (%)			Time (s)			Nodes	
		Average	Standard deviation	Maximum	Average	Standard deviation	Maximum	Average	Standard deviation
$n = 40$	6.41	0.83	0.57	1.70	36.43	33.84	139.67	935.23	889.69
$n = 60$	7.53	1.40	0.56	3.18	164.8	228.29	1,191.09	1,049.53	1,391.35
$n = 80$	11.36	1.61	0.77	3.24	435.53	414.92	2,075.78	1,070.37	1,038.47
$n = 100$	12.29	1.87	0.92	4.71	1,592.50	1,570.65	6,915.30	1,193.97	1,241.37
$n = 150$	17.33	2.42	1.37	2.97	2,151.02	1,192.68	4,630.32	573.60	67.24
$n = 200$	18.61	2.53	1.03	3.82	5,724.67	2,807.91	11,058.63	703.60	214.19
$OS = 0.1$	8.58	1.56	1.07	4.20	121.60	92.56	412.33	896.60	812.99
$OS = 0.2$	8.43	1.26	0.73	3.16	130.04	88.14	448.90	881.70	657.08
$OS = 0.3$	9.65	1.47	0.73	3.05	137.56	125.30	735.66	856.20	956.04
$OS = 0.4$	11.59	1.00	0.55	1.81	162.20	176.51	745.02	1,133.93	1,390.46
$OS = 0.5$	11.30	1.19	0.71	3.60	120.66	94.66	583.08	788.90	685.91
$a_{1i} \sim U[0.15, 0.45]$	10.16	1.43	0.77	3.28	175.88	233.77	1,297.63	1,173.63	1,759.80
$a_{1i} \sim U[0.10, 0.50]$	10.46	1.49	0.84	3.67	159.20	151.97	638.67	1,027.40	1,062.15
$a_{1i} \sim U[0.05, 0.55]$	9.78	1.36	0.69	3.36	130.48	121.79	684.24	850.93	1,016.47
$c_i \sim U[0.04, 0.08]$	9.20	1.30	0.91	3.83	170.79	251.41	945.03	1,313.93	2,087.47
$c_i \sim U[0.02, 0.10]$	9.94	1.31	0.66	3.12	151.68	140.77	603.97	989.00	1,077.43
$c_i \sim U[0.00, 0.12]$	9.88	1.50	1.00	4.95	141.16	129.76	653.64	888.90	927.97
$m_i \sim U[0.2, 0.9]$	8.94	1.21	0.63	2.39	147.93	194.21	1,021.78	976.37	1,249.55
$m_i \sim U[0.1, 1.0]$	9.28	1.22	0.83	3.70	131.18	146.73	810.04	905.79	1,102.77
$m_i \sim U[0.0, 1.1]$	9.25	1.12	0.81	3.49	112.56	109.12	609.56	782.93	859.12
$q_{ji} \sim U[0.30, 0.70]$	8.42	1.23	0.65	2.57	215.82	485.16	2,742.49	1,462.53	3,296.40
$q_{ji} \sim U[0.15, 0.85]$	8.88	1.32	0.60	3.01	192.29	275.62	1,389.50	1,325.63	2,104.87
$q_{ji} \sim U[0.00, 1.00]$	9.01	1.41	0.82	3.44	131.75	207.53	1,221.07	877.40	1,462.35

strength (OS) (Mastor 1970). The OS is the number of precedence relations (including transitive ones but excluding arcs incident to the dummy start and end tasks) divided by the theoretical maximum number of precedence relations, which is $n(n-1)/2$.

We first investigate the effect of project size on algorithm performance, using instances with $OS = 0.3$. We let the number of tasks be $n \in \{40, 60, 80, 100\}$, with the other parameters set as previously described. We generate $4 \times 30 = 120$ instances to test the effect of project size n and find optimal solutions. We also consider five instances each when $n = 150$ and $n = 200$, where we set a tolerance level of 0.5% to allow all instances to solve within reasonable time. We report the relative percentage difference, denoted as Cost_Red., between the optimal cost with and without intraproject learning. We also report the initial gap (IG), which is the relative percentage difference between the upper and lower bounds before branching, and the running time. The IG measures the accuracy of our heuristic procedure, Algorithm DI, described in Section 5.

We discuss the results in Table 4 related to project size. From the Cost_Red. column, the reductions in objective value from learning consistently increase with project size n , to about 20% for $n = 200$. They are also consistent across the various parameters shown. We are thus able to demonstrate substantial and robust benefit in project cost as a result of intraproject learning. The results in the IG column, which increase in n but at a decreasing speed, show that the initial gap between the

upper and lower bounds is typically less than 2% for $n \leq 100$. These small values of IG confirm the accuracy of Algorithm DI, the heuristic in Section 5, which can be used to obtain quick solutions. The Time and Nodes columns indicate that the problem can be optimally solved quickly when the number of tasks is not more than 100 and can be solved with high precision within reasonable time for up to 200 tasks.

Next, we study the effect of the density of precedence constraints OS, knowledge response a_{1i} , unit time learning cost c_i , unit time resource consumption rate m_i , and learning rate matrix q_{ji} on the performance of our solution procedures. Let $n = 60$ for all these tests. We consider five values of OS, and three ranges from which the a_{1i} , c_i , m_i , and q_{ji} values are generated while keeping the mean constant. Table 4 shows the OS values and ranges. For each setting, we generate 30 networks from RanGen1, giving 17 scenarios and 510 random instances. The results also appear in Table 4.

Starting from row entry “OS = 0.1...” of Table 4, we observe that the average initial gap is less than 2% for all the scenarios, which once again shows the accuracy and robustness of Algorithm DI. We also see that the OS of a network has a significant influence on the running time “Time.” The running time increases in OS up to 0.4 and thereafter decreases. For larger OS values in this range, learning can be applied to many different tasks early in the project. Hence, it is more time-consuming to solve each branch, which causes the increase in running time while $OS \leq 0.4$.

Changes in the knowledge response coefficient a_{li} , learning cost coefficient c_i , resource consumption coefficient m_i , and learning rate matrix q_{ji} influence the running time significantly: The greater is the variance of the coefficients, the shorter is the running time. The larger variance of the coefficients results in greater variance in the objective function values on different branches, which enables removal of more branches early in the branch-and-bound process. Overall, these results suggest that our work can be useful to managers of projects with a wide variety of characteristics.

7.2. Sensitivity to Misestimation of Learning Rate Matrix and Learning Function

Most of the data used in our model are standard within project management. However, estimation of the task learning rate matrix q_{ji} and learning function $f_i(\cdot)$ parameters is nonstandard. Hence, we provide a sensitivity analysis to evaluate the effect of any misestimation. For every parameter $X \in \{q_{ji}, a_{li}\}$, we use X_{true} to represent the true value of parameter X , and X_{est} to represent the estimated value of X in our model, respectively. Then, we use X_{est} as data and find the corresponding optimal solution with cost Obj_{est} , where the solution evaluated using true parameters X_{true} has cost Obj_{true} . We consider two performance metrics for this error, defined as follows:

$$\kappa_1 = \frac{Obj_{true} - Obj_{est}}{Obj_{est}} \times 100\%, \quad (51)$$

$$\begin{aligned} \kappa_2 &= \frac{(Obj_{no_learning} - Obj_{true}) - (Obj_{no_learning} - Obj_{est})}{Obj_{no_learning} - Obj_{est}} \\ &\times 100\% = \frac{Obj_{est} - Obj_{true}}{Obj_{no_learning} - Obj_{est}} \times 100\%. \end{aligned} \quad (52)$$

Here, $Obj_{no_learning}$ represents the optimal cost without learning. Therefore, κ_1 measures the percentage difference between the actual cost of using X_{true} and the estimated optimal cost using X_{est} , whereas κ_2 measures the percentage difference between the actual benefit gained from learning and the estimated benefit.

Our sensitivity analysis uses eight different parameter combinations of project size n , order strength OS , and serial/parallel indicator $I2$ (Tavares et al. 1999).¹ For each combination, we generate 30 random networks using RanGen1 and RanGen2. All the parameters are generated as specified at the beginning of Section 7. To model the estimation error, for each one of the parameters $X \in \{q_{ji}, a_{li}\}$, we assume that its percentage estimation error $\rho_X \sim N(0, 0.15^2)$ such that $X_{true} = (1 + \rho_X) \cdot X_{est}$, where the q_{ji} values are truncated at zero and one. For each instance, we simulate 5,000 sets of the true parameters and calculate κ_1 and κ_2 . Hence, each combination contains 150,000 sample points. The results in Table 5 include the percentage mean, standard deviation, 10th percentile, and 90th percentile for κ_1 and κ_2 .

From Table 5, the standard deviation of the percentage estimation errors is relatively large at 15%, which induces only small cost errors. The average κ_1 is not more than 0.94%, and the average κ_2 is not smaller than -7.31% . Moreover, in 90% of the cases, κ_1 is smaller than 1.53% and κ_2 is larger than -18.31% , for any combination. Thus, we demonstrate significant value from the use of an optimization approach to intraproject learning in Section 7.1, and further conclude that the benefits of our model are robust against parameter misestimation.

7.3. Managerial Insights from Controlled Experiments

In this section, we use controlled experiments and regression analysis to obtain useful managerial insights for intraproject learning. Two main questions are addressed:

1. At the project level, how do factors including the project size n , the serial/parallel indicator $I2$, and the order strength OS influence the effectiveness of intraproject learning?

2. At the task level, what are the factors that influence priorities for investment in learning?

To answer these, we generate instances with different combinations of factors, solve them with our algorithms, and perform regression analysis to infer the influence of the factors on the performance of intraproject learning. We use three levels of project size (20, 50,

Table 5. Sensitivity to Parameter Misestimation

n	Parameters ($OS/I2$)	κ_1 (%)				κ_2 (%)			
		Mean	Standard deviation	p10	p90	Mean	Standard deviation	p10	p90
40	$OS = 0.3$	0.5	0.67	-0.26	1.38	-6.7	9.12	-18.31	3.63
40	$OS = 0.7$	0.44	0.49	-0.14	1.08	-4.91	5.39	-11.83	1.58
80	$OS = 0.3$	0.94	0.74	0.09	1.9	-7.31	5.83	-14.79	-0.75
80	$OS = 0.7$	0.83	0.55	0.16	1.53	-5.24	3.48	-9.7	-1.07
40	$I2 = 0.3$	0.43	0.49	-0.16	1.05	-3.89	4.41	-9.49	1.57
40	$I2 = 0.7$	0.38	0.35	-0.05	0.82	-3.47	3.22	-7.56	0.5
80	$I2 = 0.3$	0.63	0.45	0.08	1.2	-3.19	2.24	-6.06	-0.42
80	$I2 = 0.7$	0.51	0.29	0.14	0.89	-2.61	1.48	-4.51	-0.74

and 80), three levels of $I2$ (0.3, 0.5, and 0.7), three levels of OS (0.3, 0.5, and 0.7), and in total 24 different $n \times I2 \times OS$ combinations.² For each $n \times I2 \times OS$ combination, we generate 50 random networks using RanGen2 (Demeulemeester et al. 2003). Therefore, we have $24 \times 50 = 1,200$ random instances.

7.3.1. Project Size and Network Structure. We first consider the following regression model, where subscript m represents “medium” and l represents “large”:

$$\text{Cost_Red}_j = \rho_0 + \rho_1 N_{m_j} + \rho_2 N_{l_j} + \rho_3 I2_{m_j} + \rho_4 I2_{l_j} + \rho_5 OS_{m_j} + \rho_6 OS_{l_j} + \phi \mathbf{X}_j + \mu_j, \quad (53)$$

where Cost_Red_j represents the cost reduction percentage of instance j when performing optimal intraproject learning, $N_{m_j} = \mathbb{1}\{n = 50\}$ and $N_{l_j} = \mathbb{1}\{n = 80\}$ are project size indicators, $I2_{m_j} = \mathbb{1}\{I2_j = 0.5\}$ and $I2_{l_j} = \mathbb{1}\{I2_j = 0.7\}$ are $I2$ indicators, $OS_{m_j} = \mathbb{1}\{OS_j = 0.5\}$ and $OS_{l_j} = \mathbb{1}\{OS_j = 0.7\}$ are order strength indicators, and μ_j is the error term; $\mathbf{X}_j$ are the control variables, including the average resource consumption rate, the average learning cost coefficient, the average knowledge response, and the average of all the elements of the learning rate matrix of instance j .

The regression results from (53) are presented in Table 6. The coefficients of the project size indicators are both significantly positive ($\hat{\rho}_1 = 0.077$ and $\hat{\rho}_2 = 0.123$), and their magnitude is increasing in the project size n . Therefore, in general, large size projects have greater potential benefit from intraproject learning.

The same pattern can be observed for the coefficients of the $I2$ indicator ($\hat{\rho}_3 = 0.009$ and $\hat{\rho}_4 = 0.016$), which implies that when holding OS fixed, the benefit from learning increases as the $I2$ level increases, that is, the shape of the project network becomes closer to serial. An explanation is that, given the same density of precedence constraints, the increased project length enlarges the initial learning sets for many tasks, which makes knowledge less costly to use because learning is applied

to noncritical tasks and thus provides a greater incentive to learn.

However, for the order strength, the cost reduction gained by intraproject learning decreases in OS ($\hat{\rho}_5 = -0.012$, $\hat{\rho}_6 = -0.025$) when holding the $I2$ level fixed. This is because with higher OS , the project network is less flexible in changing the learning set for each task, that is, the size of the learning collection Ω is shrunk, which limits the learning potential.

From the previous results for OS and $I2$, the learning benefit is largest in networks with large $I2$ and small OS level. However, large $I2$ tends to imply high OS (recall that $OS \geq (I2)^2 - \frac{1}{4(N-1)}$). For example, when $I2 = 1$, OS can only be one, and the network is a chain. Thus, the learning benefit gained from the large $I2$ is offset by the large OS level. Therefore, the benefit from learning is largest in networks with midrange $I2$ level and smallest possible OS level. In particular, simple network structures, that is, the serial ($OS = I2 = 1$) and parallel ($OS = I2 = 0$) networks, provide the smallest potential benefit from intraproject learning.

7.3.2. Task Position and Learning Rates. To answer the task level question, we define three task sets that represent the relative task positions in a project: let $\Phi_1 = \{i \in N \mid \overline{EF}_i / C_{max} \leq 25\%\}$ be the set of earlier tasks, $\Phi_2 = \{i \in N \mid 25\% < \overline{EF}_i / C_{max} \leq 75\%\}$ be the set of middle tasks, and $\Phi_3 = \{i \in N \mid \overline{EF}_i / C_{max} > 75\%\}$ be the set of later tasks, where C_{max} is the makespan without learning and $\overline{EF}_i$ is the early finish time of task i ; see Lemma EC.6 and the related discussions in the online appendix for details of $\overline{EF}_i$.

For the task level analysis, we consider the learning percentage, “Learn_Pct,” defined by $\frac{y_i}{y_i^*}$, as the dependent variable of task i and use the following regression model:

$$\begin{aligned} \text{Learn_Pct}_{ij} = & \theta_0 + \theta_1 \text{Critical}_{ij} + \theta_2 \text{Stage}_{2ij} + \theta_3 \text{Stage}_{3ij} \\ & + \theta_4 \text{Critical}_{ij} \cdot \text{Stage}_{2ij} + \theta_5 \text{Critical}_{ij} \\ & \cdot \text{Stage}_{3ij} + \theta_6 \text{avg_q_out}_{ij} + \theta_7 n_out_{ij} \\ & + \psi \mathbf{X}_{ij} + \xi_{ij}, \end{aligned} \quad (54)$$

where Critical_{ij} is an indicator of a task i that is critical (i.e., on the longest path of the original network without learning) in instance j , $\text{Stage}_{2ij} = \mathbb{1}\{i \in \Phi_2\}$ and $\text{Stage}_{3ij} = \mathbb{1}\{i \in \Phi_3\}$ are task position indicators, n_out_{ij} represents the number of potential tasks that may benefit from knowledge generated by task i in instance j , and avg_q_out_{ij} represents the average value of the learning rate parameter $q_{i'}$, where i' represents the potential task that may benefit from the knowledge generated by task i . We use n_out_{ij} and avg_q_out_{ij} to measure the potential learning value of a task. $\mathbf{X}_{ij}$ are

Table 6. Regression Results of (53)

	Cost_Red	
	Estimates	Standard error
N_m	0.077***	(0.0014)
N_l	0.123***	(0.0015)
$I2_m$	0.009***	(0.0011)
$I2_l$	0.016***	(0.0013)
OS_m	−0.012***	(0.0013)
OS_l	−0.025***	(0.0013)
R^2	0.91	
Observations	1,200	

Notes. Standard errors in parentheses.

* $p < 0.10$; ** $p < 0.05$; *** $p < 0.01$.

Table 7. Regression Results of (54)

	Learn_Pct	
	Estimates	Standard error
Critical task	−0.47***	(0.0043)
Middle task	−0.29***	(0.0043)
Later task	−0.439***	(0.0071)
Critical task × Middle task	0.206***	(0.0055)
Critical task × Later task	0.453***	(0.007)
avg_q_out	0.462***	(0.0203)
n_out	0.007***	(0.0002)
R ²	0.534	
Observations	60,000	

Note. Standard errors in parentheses.

* $p < 0.10$; ** $p < 0.05$; *** $p < 0.01$.

the control variables, including project size, network structure parameters OS , $I2$, task i 's nominal processing time, resource consumption rate and learning cost. Also, ξ_{ij} is the error term.

Table 7 summarizes the task level regression results. As shown there, the coefficient of critical task is significantly negative ($\hat{\theta}_1 = -0.47$). This suggests that, compared with the critical tasks, *the project manager should pay more attention to learning activities for noncritical tasks*. The reasoning behind this is that critical tasks are more costly to learn because doing so directly increases the makespan, whereas we can add learning to noncritical tasks without initially delaying the project.

The relative position of the task in the project network also affects the learning percentage. For noncritical tasks, learning benefits decrease along the task stage ($\hat{\theta}_2 = -0.29$ and $\hat{\theta}_3 = -0.439$), which implies that *noncritical learning activities at the earlier stage are more beneficial and should be more extensive*. However, the effect of task position for critical tasks is weaker ($\hat{\theta}_2 + \hat{\theta}_4 = -0.084$ and $\hat{\theta}_3 + \hat{\theta}_5 = 0.014$). Furthermore, the coefficient of the learning rate index is significantly positive ($\hat{\theta}_6 = 0.462$), where tasks with higher learning rates deserve greater investment in learning. Learning should also be more extensive when the task has more potential successors, where knowledge gained can be used repetitively ($\hat{\theta}_7 = 0.007$).

7.3.3. Summary of Managerial Insights. We find consistent evidence that larger projects benefit more from intraproject learning. Furthermore, our computational studies provide the following insights.

1. The benefit from learning increases in the $I2$ level up to a point, decreases in the OS level and is largest in networks with midrange $I2$ level and smallest possible OS level. Compared with simple serial or parallel networks, projects with more complex networks possess greater potential benefit from intraproject learning.

2. Noncritical learning activities at an earlier stage are more beneficial and should be learned more extensively.

3. Tasks with higher learning rates deserve more learning investment. Learning should also be invested more in tasks with more potential successors, where knowledge gained can be used repetitively.

8. Concluding Remarks

Learning has been extensively studied within continuous, high-volume manufacturing environments. There have also been several studies of learning between projects. By contrast, the value of learning within a project, which applies the same learning strategies as interproject learning, has not been studied from an optimization perspective. Our work is apparently the first to model and solve the problem of optimizing intraproject learning. Our results confirm the substantial value of intraproject learning, as suggested but not formally modeled by Kotnour (2000) and Darter (2014). We consider the general problem of minimizing a piecewise linear, increasing, and convex objective cost function of a project. The algorithm we describe finds optimal learning strategies for projects of practical size. Moreover, the heuristic we describe for generating feasible solutions can be used as a simple standalone procedure and delivers solutions that are typically very close to optimal. Implementation of our model for a practical project at Huawei Corporation shows that it delivers significant improvement in project performance.

Our analysis of the problem enables us to provide some general insights that are of value to project managers, besides the specific ones discussed in Section 7.3.3. First, the tradeoff between too little and too much learning is clarified at multiple points in our work. Second, it is intuitively the case that investment in learning is more worthwhile at the start of the project, but our analysis defines more precisely how this value changes during project execution. Third, although it may seem counterintuitive to delay the start of an available task, we identify situations where, due to the imminent arrival of relevant lessons learned, it is indeed worthwhile to do so. Fourth, because of their different characteristics, some tasks within a project are more proficient at generating knowledge, whereas others are more proficient at using accumulated knowledge to reduce their own task times; our work clarifies the tradeoffs that arise in the relative sequencing of such tasks. Finally, we identify projects with midrange $I2$ value and smallest possible OS value as being those with the greatest potential benefit from intraproject learning.

Our work also suggests several issues that require further study. First, although our model emphasizes nonrenewable resource constraints that limit the amount of learning, it would also be valuable to include renewable resource constraints. Second, the assumption that knowledge gain is a linear function of time spent in learning can be modified; in some practical cases this

may be a concave function. Third, the assumption that task times are piecewise linear and convex functions of knowledge can be modified; in some practical situations they may be step functions. Fourth, it would be useful to generalize our model to consider the crashing of tasks and include the related costs in the objective function. Fifth, the extensive use of learning could lead to the elimination of potentially useful slack when task times are stochastic. This can be prevented by penalizing learning to maintain sufficient slack, but the detailed design of such a procedure requires further study. Sixth, although we describe a simple heuristic approach and validate its performance computationally, the design and evaluation of additional heuristics is encouraged. Seventh, although our analysis of intraproject learning assumes deterministic task times, we anticipate that a similar analysis using stochastic task times would demonstrate reductions in not only the expectations of task times but also their variance. Finally, especially for companies with many high learning rate projects, the interaction between intraproject learning and interproject learning should be studied. We hope that our work will stimulate research on these interesting and practically important topics.

Acknowledgments

The authors thank the department editor, associate editor, and two anonymous referees for valuable comments and suggestions that improved this paper.

Endnotes

¹ $I2$ measures the closeness of a network to a serial one (a chain of tasks, $I2 = 1$) or a parallel one ($I2 = 0$).

² Because OS is lower bounded by $\frac{I2 \cdot N(I2 \cdot N - 1)}{N(N-1)} = (I2)^2 - I2 \cdot \frac{1-I2}{N-1} \geq (I2)^2 - \frac{1}{4(N-1)}$, OS cannot be 0.3 when $I2$ equals 0.7 and $N \geq 20$. Thus, in total, we have $3 \cdot 3 \cdot 3 - 3 = 24$ combinations.

References

- Adler PS, Clark KB (1991) Behind the learning curve: A sketch of the learning process. *Management Sci.* 37(3):267–281.
- Ayas K (1996) Professional project management: A shift toward learning and a knowledge creating structure. *Internat. J. Project Management* 14(3):131–136.
- Ayas K, Zeniuk N (2001) Project-based learning: Building communities of reflective practitioners. *Management Learn.* 32(1):61–76.
- Biskup D (2008) A state-of-the art review on scheduling with learning effects. *Eur. J. Oper. Res.* 188(2):315–329.
- Bower DC, Walker DHT (2007) Planning knowledge for phased roll-out projects. *Project Management J.* September:45–60.
- Brucker P, Knust S, Schoo A, Thiele O (1998) A branch-and-bound algorithm for the resource-constrained project scheduling problem. *Eur. J. Oper. Res.* 107(2):272–288.
- Brucker P, Drexel A, Möhring R, Neumann K, Pesch E (1999) Resource-constrained project scheduling: Notation, classification, models, and methods. *Eur. J. Oper. Res.* 112(1):3–41.
- Christofides N, Alvarez-Valdés R, Tamarit JM (1987) Project scheduling with resource constraints: A branch and bound approach. *Eur. J. Oper. Res.* 29(3):262–273.
- Consult Yourself (2015) Passion & action. Accessed September 4, 2022, <http://www.consultyourself.nl/belief-mission-vision-2.html>.
- Darter K (2014) Documenting lessons learned: What have you learned? Accessed September 4, 2022, <https://www.projectsma rt.co.uk/documenting-lessons-learned-what-have-you-learned.php>.
- Demeulemeester E, Herroelen W (1992) A branch-and-bound procedure for the multiple resource-constrained project scheduling problem. *Management Sci.* 38(12):1803–1818.
- Demeulemeester E, Vanhoucke M, Herroelen W (2003) RanGen: A random network generator for activity-on-the-node networks. *J. Scheduling* 6(1):17–38.
- Hall NG (2016) Research and teaching opportunities in project management. *INFORMS Tutorials in Operations Research*, 329–388.
- Hartmann S, Briskorn D (2022) An updated survey of variants and extensions of the resource-constrained project scheduling problem. *Eur. J. Oper. Res.* 297:1–14.
- Hill A, Tickitt J, Vossen WM (2021) A computational study of constraint programming approaches for resource-constrained project scheduling with autonomous learning effects. *Proc. Conf. on Integration of Constraint Programming, Artificial Intelligence, and Oper. Res.: 18th Internat. Conf. CPAIOR 2021, Proceedings 18* (Springer, New York), 26–44.
- Hu X, Cui N, Demeulemeester E (2015) Effective expediting to improve project due date and cost performance through buffer management. *Internat. J. Production Res.* 53(5):1460–1471.
- Julian J (2008) How project management office leaders facilitate cross-project learning and continuous improvement. *Project Management J.* 39(3):43–58.
- Kerzner H (2013) *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*, 11th ed. (Wiley, Hoboken, NJ).
- Klasterin TD (2012) *Project Management: Tools and Trade-Offs*, 3rd ed. (Pearson).
- Kolisch R, Hartmann S (2006) Experimental investigation of heuristics for resource-constrained project scheduling: An update. *Eur. J. Oper. Res.* 174(1):23–37.
- Kotnour T (2000) Organizational learning practices in the project management environment. *Internat. J. Qual. Reliability Management* 17(4/5):393–406.
- Lapr  MA, Mukherjee AS, Van Wassenhove LN (2000) Behind the learning curve: Linking learning activities to waste reduction. *Management Sci.* 46(5):597–611.
- Levy FK (1965) Adaptation in the production process. *Management Sci.* 11(6):B136–B154.
- Li Z, Zhou S, Wan G (2022) Replenishment management and inventory optimization in a large international consumer electronics supply chain. *J. Systems Management* 31(6):1075–1083.
- Mangematin V, Blanco S, Deschamp B, Genet C, Kahane B (2006) Project management: Learning by breaking the rules. Working Papers 2006-06. Grenoble Applied Economics Laboratory (GAEL), INRA – Universit  Pierre Mend s, Grenoble, France.
- March JG (1991) Exploration and exploitation in organizational learning. *Organ. Sci.* 2(1):71–87.
- Master AA (1970) An empirical investigation and comparative evaluation of production line balancing techniques. *Management Sci.* 16(11):728–746.
- Mingozzi A, Maniezzo V, Ricciardelli S, Bianco L (1998) An exact algorithm for the resource-constrained project scheduling problem based on a new mathematical formulation. *Management Sci.* 44(5):714–729.
- M hring RH, Schulz AS, Stork F, Uetz M (2003) Solving project scheduling problems by minimum cut computations. *Management Sci.* 49(3):330–350.
- Prencipe A, Tell F (2001) Inter-project learning: Processes and outcomes of knowledge codification in project-based firms. *Res. Policy* 30(9):1373–1394.

- Rice MP, Colarelli O'Connor G, Pierantozzi R (2008) Implementing a learning plan to counter project uncertainty. *MIT Sloan Management Rev.* 49(2):54–62.
- Schindler M, Eppler MJ (2003) Harvesting project knowledge: A review of project learning methods and success factors. *Internat. J. Project Management* 21(3):219–228.
- Sense AJ (2007) Structuring the project environment for learning. *Internat. J. Project Management* 25(4):405–412.
- Shipton H, Dawson J, West M, Patterson M (2002) Learning in manufacturing organizations: What factors predict effectiveness? *Human Resources Development Internat.* 5(1):55–72.
- Simplilearn (2015) Best practices for preparing a lessons learned document. Accessed September 4, 2022, <http://www.simplilearn.com/practices-for-preparing-a-lessons-learned-document-article>.
- Tavares LV, Ferreira JA, Coelho JS (1999) The risk of delay of a project in terms of the morphology of its network. *Eur. J. Oper. Res.* 119(2):510–537.
- Turner JR, Keegan AE, Crawford L (2000) Learning by experience in the project-based organization. *Proc. PMI Res. Conf.* (Project Management Institute, Sylva, NC).
- Van Peteghem V, Vanhoucke M (2015) Influence of learning in resource-constrained project scheduling. *Comput. Industrial Engrg.* 87:569–579.
- Von Zedtwitz M (2002) Organizational learning through post-project-reviews in R&D. *R&D Management* 32(3):255–268.
- Wang J, Sun L, Sun L (2010) Scheduling jobs with an exponential sum-of-actual-processing-time-based learning effect. *Comput. Math. Appl.* 60(9):2673–2678.
- Wang J-B, Lv D-Y, Xu J, Ji P, Li F (2021) Bicriterion scheduling with truncated learning effects and convex controllable processing times. *Internat. Trans. Oper. Res.* 28:1573–1593.
- Williams T (2008) How do organizations learn lessons from projects: And do they? *IEEE Trans. Engrg. Management* 55(2):248–266.
- Zollo M, Reuer JJ (2010) Experience spillovers across corporate development activities. *Organ. Sci.* 21(6):1195–1212.

Online Appendix

Proof of Theorem 1. By reduction from the following problem that is known (Garey and Johnson, 1979, p.221) to be unary *NP*-complete:

Exact Cover by 3-Sets: given a set X with $|X| = 3\nu$ and a collection C of 3-element subsets of X , does C contain an exact cover for X , i.e., a subcollection $C' \subseteq C$ such that every element of X occurs in exactly one member of C' ?

Given an arbitrary instance of Exact Cover by 3-Sets, we construct an instance of Problem (P) as defined below. We index the tasks corresponding to the 3-element subsets of X as $1, \dots, |C|$, and the tasks corresponding to the elements of X as $|C| + 1, \dots, |C| + 3\nu$. Three additional tasks are defined. We denote the minimum possible duration of task i , when maximally reduced by learning from other tasks, as $\underline{p}_i$, $i = 1, \dots, n$.

- The number of tasks: $n = |C| + 3\nu + 3$.
- Nominal and minimum durations of task i :

$$\begin{cases} t_i = \underline{p}_i = \Lambda, & i = 1, \dots, |C|; \\ t_i = 2\Lambda, \underline{p}_i = \Lambda - 0.5, & i = |C| + 1, \dots, |C| + 3\nu; \\ t_i = \underline{p}_i = \Lambda + 0.5 & i = |C| + 3\nu + 1; \\ t_i = \underline{p}_i = \Lambda + 1 & i = |C| + 3\nu + 2; \\ t_i = \Lambda + |C| - \nu/2 - 1, \underline{p}_i = \Lambda - 1, & i = |C| + 3\nu + 3, \end{cases}$$

where $\Lambda \geq |C|$ is an integer.

- The maximum deliberate learning duration of task i :

$$\bar{y}_i = \begin{cases} 1, & i = 1, \dots, |C|, \\ 0, & \text{otherwise.} \end{cases}$$
- Autonomous learning: $\delta_{ji} = 0$, $i, j = 1, \dots, n$.
- Learning rate between task i and task j :

$$q_{ji} = \begin{cases} 1, & \text{if the } j\text{th subset of } C \text{ contains element } i, 1 \leq j \leq |C|, |C| + 1 \leq i \leq |C| + 3\nu, \\ 1, & \text{if } 1 \leq j \leq |C|, i = |C| + 3\nu + 3, \\ 0, & \text{otherwise.} \end{cases}$$
- The learning function:

$$f_i(\sum_{j \in A_i} q_{ji} y_j) = \begin{cases} \min\{(2\Lambda + 1) \sum_{j \in A_i} q_{ji} y_j, t_i - \underline{p}_i\}, & i \in \{|C| + 1, \dots, |C| + 3\nu\}, \\ \min\{\sum_{j \in A_i} q_{ji} y_j, t_i - \underline{p}_i\}, & i = |C| + 3\nu + 3 \\ 0, & \text{otherwise.} \end{cases}$$

- $\mathbf{d}^t$ is sufficiently large to make Constraints (8) redundant;
- The extended set of precedence constraints:

$$E^T = (\{|C| + 3\nu + 1\} \rightarrow \{|C| + 1, \dots, |C| + 3\nu\}) \cup (\{|C| + 3\nu + 2\} \rightarrow \{|C| + 3\nu + 3\}).$$

- $M = 2\Lambda$: the scheduled makespan of the project.
- $Z = \max\{C_{\max} - M, 0\}$: the makespan penalty, which is also the total project cost.

In this constructed instance of Problem (P), tasks $1, \dots, |C|$ represent the $|C|$ given subsets (with three elements) in the instance of Exact Cover by 3-Sets. Tasks $|C| + 1, \dots, |C| + 3\nu$ represent the 3ν given elements of the set X in that instance. Task $|C| + 3\nu + 1$ represents an additional task that precedes tasks $|C| + 1, \dots, |C| + 3\nu$ in the project network. Finally, tasks $|C| + 3\nu + 2$ and $|C| + 3\nu + 3$ form a chain in the project network that spans the makespan of the project.

We show that there exists a solution to the constructed instance of Problem (P) with $Z = 0$ if and only if there exists an Exact Cover by 3-Sets.

($\Rightarrow$) Suppose there exists an Exact Cover by 3-Sets.

First, schedule task $|C| + 3\nu + 1$ to start at time 0 and complete processing at time $\Lambda + 0.5$. Schedule task $|C| + 3\nu + 2$ to start at time 0 and complete processing at time $\Lambda + 1$.

Next, number the subsets in C such that subsets $1, \dots, \nu$ form an Exact Cover by 3-Sets. Schedule the corresponding tasks $1, \dots, \nu$ to start at time 0, and complete processing at time Λ . Let $y_i = 0.5$, for $i = 1, \dots, \nu$. Observe that tasks $1, \dots, \nu$ form an Exact Cover by 3-Sets with tasks $|C| + 1, \dots, |C| + 3\nu$. Consequently, when we start tasks $|C| + 1, \dots, |C| + 3\nu$ at time $\Lambda + 0.5$, those tasks each have available $(2\Lambda + 1) * (0.5) = \Lambda + 0.5$ units of learning from the previously completed tasks $1, \dots, \nu$. Hence, tasks $|C| + 1, \dots, |C| + 3\nu$ all have their durations reduced by $\Lambda + 0.5$, and all complete at time $\Lambda + 0.5 + [2\Lambda - (\Lambda + 0.5)] = 2\Lambda = M$.

Finally, schedule tasks $\nu + 1, \dots, |C|$ to start at time 0 and complete processing at time Λ , followed by learning time $y_i = 1 = \bar{y}_i$. The completed tasks $1, \dots, |C|$ collectively make $|C| - \nu/2$ units of learning available to task $|C| + 3\nu + 3$ at time $\Lambda + 1$. Then, schedule task $|C| + 3\nu + 3$ to start at time $\Lambda + 1$, with its duration reduced by $|C| - \nu/2$, and complete processing at time $\Lambda + 1 + [(\Lambda + |C| - \nu/2 - 1) - (|C| - \nu/2)] = 2\Lambda = M$. Hence, $Z = \max\{C_{\max} - M, 0\} = 0$.

The constructed schedule is shown in Figure EC.1. In this figure, we use $\{i, \dots, j\}$ to denote a set of tasks that are all processed concurrently with the same start and completion time shown. We use $\mathcal{L}$ to denote time spent learning. Precedence constraints are denoted by bold arrows. The transfer of learning is denoted by triple parallel arrows.

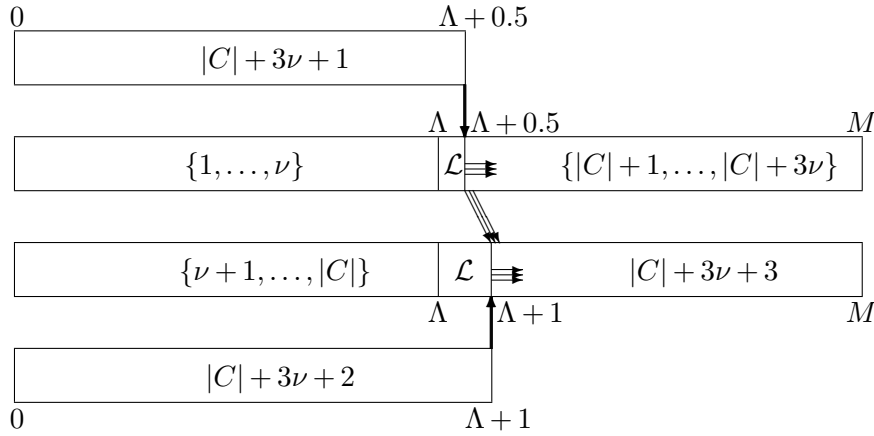
($\Leftarrow$) Suppose there exists a schedule for the constructed instance of Problem (P) with $Z = 0$.

We first describe several necessary properties of such a schedule.

Property 1. (a) Since $\{|C| + 3\nu + 1\} \rightarrow \{|C| + 1, \dots, |C| + 3\nu\}$, tasks $|C| + 1, \dots, |C| + 3\nu$ cannot start before time $\Lambda + 0.5$. (b) Similarly, since $\{|C| + 3\nu + 2\} \rightarrow \{|C| + 3\nu + 3\}$, tasks $|C| + 3\nu + 3$ cannot start before time $\Lambda + 1$.

Property 2. Since tasks $1, \dots, |C|$ have nonreducible durations and no predecessor tasks, without loss of generality they can all be scheduled to start at time 0 and complete processing at time Λ .

Property 3. From Property 1(a), task $|C| + 1, \dots, |C| + 3\nu$ cannot start before time $\Lambda + 0.5$. Since $M - (\Lambda + 0.5) = \Lambda - 0.5$ equals their minimum duration after learning from other tasks, they must

Figure EC.1 Constructed Schedule with $Z = 0$, Given an Exact Cover by 3-Sets

all start at time $\Lambda + 0.5$ and achieve their maximum learning from other tasks, else either Property 1(a) does not hold or $Z > 0$.

Property 4. From Property 1(b), task $|C| + 3\nu + 3$ cannot start before time $\Lambda + 1$. Since $M - (\Lambda + 1) = \Lambda - 1$ equals its minimum duration after learning from other tasks, it must start at time $\Lambda + 1$ and achieve its maximum learning from other tasks, else either Property 1(b) does not hold or $Z > 0$.

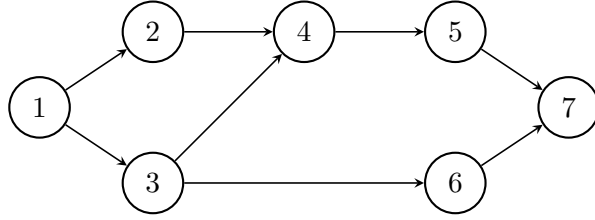
Property 5. From Property 4, the learning that task $|C| + 3\nu + 3$ must receive by time $\Lambda + 1$ is at least $t_{|C|+3\nu+3} - p_{|C|+3\nu+3} = |C| - 0.5\nu$. Further, (a) task $|C| + 3\nu + 3$ can only learn from tasks $1, \dots, |C|$; (b) $\bar{y}_i = 1$; and (c) $q_{ji} = 1$. Hence, at most ν tasks i can have $y_i \leq 0.5$, else task $|C| + 3\nu + 3$ cannot achieve its maximum learning of $|C| - 0.5\nu$, which contradicts Property 4.

Now, let $\zeta_1 = \{i | 1, \dots, |C|, S_i = 0, y_i = 0.5\}$, $\zeta_2 = \{i | 1, \dots, |C|, S_i = 0, y_i < 0.5\}$, and $\zeta_3 = \{i | 1, \dots, |C|, S_i = 0, y_i > 0.5\}$. Then, from Property 3, no task in ζ_3 can provide learning to any of tasks $|C| + 1, \dots, |C| + 3\nu$. Also, from Property 5, $|\zeta_1| + |\zeta_2| \leq \nu$.

If $|\zeta_1| + |\zeta_2| < \nu$, then since $|\{i | \nu_{ji} > 0, |C| + 1 \leq i \leq |C| + 3\nu\}| = 3$ for $1 \leq j \leq |C|$, not all tasks in $|C| + 1, \dots, |C| + 3\nu$ can achieve their maximum learning of $\Lambda + 0.5$ from other tasks, which contradicts Property 3. Hence, we have $|\zeta_1| + |\zeta_2| = \nu$.

Now, suppose that $|\zeta_2| > 0$. Then, the maximum amount of learning that is available from tasks in $\zeta_1 \cup \zeta_2 \cup \zeta_3 = \{1, \dots, |C|\}$ to task $|C| + 3\nu + 3$ is $\sum_{i \in \zeta_1 \cup \zeta_2 \cup \zeta_3} y_i < 0.5 * \nu + 1 * (|C| - \nu) = |C| - 0.5\nu$. Hence, task $|C| + 3\nu + 3$ cannot achieve its maximum learning ($|C| - 0.5\nu$), in which case either Property 4 is contradicted or $Z > 0$. Hence, $|\zeta_2| = 0$, which implies $|\zeta_1| = \nu$.

Further, if any task in $|C| + 1, \dots, |C| + 3\nu$ does not receive learning from one or more of the tasks of ζ_1 , Property 3 is contradicted. Hence, the learning rate coefficients q_{ji} from the tasks $j \in \zeta_1$ which represent 3-element subsets of C , to the tasks $i \in \{|C| + 1, \dots, |C| + 3\nu\}$ which represent elements of X , form an Exact Cover by 3-Sets. $\square$

Figure EC.2 Example Network**An Instance of $\bar{P}(\mathbf{A})$**

Consider the example in Figure EC.2. The corresponding Γ of the project network is: $\Gamma = \{[2, 3], [2, 6], [4, 6], [5, 6]\}$. Then, we can generate the set of learning collections Ω by assigning one of the three relationships to each pair, and consequently Ω includes $3 * 4 = 12$ feasible learning collections. As an example, consider a feasible learning collection $\mathbf{A} = \{A_1, \dots, A_7\}$, where $A_1 = \emptyset$, $A_2 = \{1\}$, $A_3 = \{1\}$, $A_4 = \{1, 2, 3\}$, $A_5 = \{1, 2, 3, 4\}$, $A_6 = \{1, 2, 3\}$, $A_7 = \{1, 2, 3, 4, 5, 6\}$. In the learning collection, task 2 is parallel with task 3 and completes before the start of task 6 (and thus can serve as a learning source of task 6), task 4 and 5 are parallel with task 6.

For the above $\mathbf{A}$, we formulate $\bar{P}(\mathbf{A})$. For simplicity, we assume that the objective is to minimize the project makespan and there are no resource constraints. Then:

$$(\bar{P}(\mathbf{A})) \quad \min_{\mathbf{S}, \mathbf{Y}} C_7 \quad (\text{EC.1})$$

$$\text{s.t.} \quad S_i - C_1 \geq 0, \quad i = 2, 3, \quad (\text{EC.2})$$

$$S_4 - C_j \geq 0, \quad j = 1, 2, 3, \quad (\text{EC.3})$$

$$S_5 - C_j \geq 0, \quad j = 1, \dots, 4, \quad (\text{EC.4})$$

$$S_6 - C_j \geq 0, \quad j = 1, 2, 3, \quad (\text{EC.5})$$

$$S_7 - C_j \geq 0, \quad j = 1, \dots, 6, \quad (\text{EC.6})$$

$$C_i - S_i = t_i - f_i\left(\sum_{j \in A_i} q_{ji} y_j\right) - g_i\left(\sum_{j \in A_i} \delta_{ji}\right) + y_i, \quad i = 1, \dots, 7. \quad (\text{EC.7})$$

Here, Constraint sets (EC.2)-(EC.6) ensure the consistency of $\mathbf{A}$, and Constraint set (EC.7) determines the actual processing times of all the tasks.

Proof of Lemma 1. In this proof, we denote $(\bar{P}(\mathbf{A}))$ without Constraint set (5) as $(\bar{P}'(\mathbf{A}))$ and the corresponding objective function as $G'(\mathbf{A})$ for notational convenience. Suppose, to the contrary, that Constraint set (5) cannot be removed. Consequently, by removing it, we will get a different solution $(\mathbf{S}^{**}, \mathbf{Y}^{**}, \mathbf{C}^{**})$ from $(\bar{P}'(\mathbf{A}^*))$ with $G'(\mathbf{A}^*) < G(\mathbf{A}^*)$, and in the solution there must exist some task pairs (i, j) with $j \notin A_i^*$ but $S_i - C_j \geq 0$. Then, by letting $A_i^{**} = A_i^* \cup \{j\}$ for all such i and j and $A_l^{**} = A_l^*$ otherwise, we get a new learning collection $\mathbf{A}^{**}$. Note that $(\mathbf{S}^{**}, \mathbf{Y}^{**})$ is feasible in $(\bar{P}(\mathbf{A}^{**}))$. Therefore, $G(\mathbf{A}^{**}) \leq H(\mathbf{C}^{**}, \mathbf{Y}^{**}) = G'(\mathbf{A}^*) < G(\mathbf{A}^*)$ and thus $(\mathbf{S}^*, \mathbf{Y}^*, \mathbf{A}^*)$ is not an optimal solution, contradiction. $\square$

Proof of Lemma 2. Suppose to the contrary that there exists a task $i \in V$ with $S_i^* \neq C_j^*$ for any $j \in V$. Let $\Phi = \{j \in V \mid C_j^* < S_i^*\}$. Construct another solution $(\bar{\mathbf{S}}^*, \bar{\mathbf{C}}^*, \mathbf{Y}^*)$ where $\bar{S}_l^* = S_l^*$ for $l \in V \setminus \{i\}$ and $\bar{S}_i^* = \max_{j \in \Phi} \{C_j^*\} < S_i^*$. Since the learning sets of tasks $j \in \{i\} \cup \Phi$ are unchanged and all the learning times are fixed, we have $\bar{C}_j^* = C_j^*$ for $j \in \Phi$ and $\bar{C}_i^* = C_i^* - S_i^* + \bar{S}_i^* < C_i^*$. For $j \in V \setminus (\{i\} \cup \Phi)$, since task i 's completion time becomes smaller, task i may become a new learning source of task j so that $\bar{C}_j^* \leq C_j^*$. Recall that we require $\Theta^C \geq \mathbf{0}$. Then, $(\bar{\mathbf{S}}^*, \bar{\mathbf{C}}^*, \mathbf{Y}^*)$ generates an objective value that is either reduced or unchanged. Hence, $(\bar{\mathbf{S}}^*, \bar{\mathbf{C}}^*, \mathbf{Y}^*)$ is also optimal. $\square$

Proof of Lemma 3. For any task j in A_i^l , the constraint set (4) ensures that $S_i \geq C_j$ in problem $P1(\mathbf{A}^l)$. Then, the optimal solution obtained satisfies this constraint, and j is contained in the updated learning set A_i^{l+1} constructed from the optimal solution. $\square$

Proof of Theorem 2. Let $(\mathbf{S}^{l*}, \mathbf{C}^{l*}, \mathbf{Y}^{l*})$ be the optimal solution of the problem $P1(\mathbf{A}^l)$. From Lemma 1, it may not be a feasible solution for problem (P) . We now construct a feasible solution $(\mathbf{S}^{l*}, \mathbf{C}^{l+1}, \mathbf{Y}^{l*})$, where $\mathbf{C}^{l+1}$ is determined based on the new constructed learning collection $\mathbf{A}^{l+1}$ and the constraint set (6). From Lemma 3, we have $A_i^l \subseteq A_i^{l+1}$ for $i \in V$. Hence, $\mathbf{C}^{l+1} \leq \mathbf{C}^{l*}$, i.e., $C_i^{l+1} \leq C_i^{l*}$, for $i \in V$, since both $f_i(\cdot)$ and $g_i(\cdot)$ are increasing functions.

Since $\Theta_i^C \geq \mathbf{0}$ for $i = 1, 2, \dots, L$, we have $\Theta_i^C \mathbf{C}^{l+1,t} + \Theta_i^Y \mathbf{Y}^{l*,t} + \beta_i \leq \Theta_i^C \mathbf{C}^{l*,t} + \Theta_i^Y \mathbf{Y}^{l*,t} + \beta_i \leq T$ for $i = 1, 2, \dots, L$ (recall that superscript t is to denote a transpose column vector). Then, from the objective function (10), we have $H(\mathbf{C}^{l+1}, \mathbf{Y}^{l*}) \leq H(\mathbf{C}^{l*}, \mathbf{Y}^{l*}) = G(\mathbf{A}^l)$. Note that $(\mathbf{S}^{l*}, \mathbf{C}^{l+1}, \mathbf{Y}^{l*})$ satisfies constraints (8) and is a feasible solution of $P(\mathbf{A}^{l+1})$. Thus, $G(\mathbf{A}^{l+1}) \leq H(\mathbf{C}^{l+1}, \mathbf{Y}^{l*}) \leq G(\mathbf{A}^l)$, and $\mathbf{A}^{l+1}$ is a better learning collection than $\mathbf{A}^l$.

Finally, since $\mathbf{A}^{l+1}$ is constructed by $(\mathbf{S}^{l*}, \mathbf{C}^{l*})$, we have $S_i^{l*} - C_j^{l*} < 0$, for $j \notin A_i^{l+1}$. When Procedure AddLearning stops, we have $\mathbf{A}^{l+1} = \mathbf{A}^l$ and $(\mathbf{S}^{l+1*}, \mathbf{C}^{l+1*}, \mathbf{Y}^{l+1*}) = (\mathbf{S}^{l*}, \mathbf{C}^{l*}, \mathbf{Y}^{l*})$. Then, $S_i^{l+1*} - C_j^{l+1*} < 0$, for $j \notin A_i^{l+1}$. Consequently, $\mathbf{A}^{l+1}$ is nested, and satisfies Constraints (5). Then, $(\mathbf{S}^{l+1*}, \mathbf{C}^{l+1*}, \mathbf{Y}^{l+1*})$ is a feasible solution to the original problem $\bar{P}(\mathbf{A})$ and (P) .

Regarding the running time of AddLearning, since there are n learning sets and each can increase in cardinality $O(n)$ times, the number of iterations at which a linear program is solved is $O(n^2)$. $\square$

Proof of Lemma 4. We assume $A_i \neq \emptyset$, else the result holds trivially. First, we show that any nonempty set $W \subset V$ cannot contain a cycle.

If $|W| < 2$, then W cannot contain a cycle, by definition. If $|W| \geq 2$, suppose there exists a cycle of tasks $j_1, j_2, \dots, j_m \in W$ such that $j_1 \in A_{j_2}$, $j_2 \in A_{j_3}, \dots, j_m \in A_{j_1}$. Then, from Definition 1, we have $C_{j_1} < C_{j_2} < C_{j_3} < \dots < C_{j_m} < C_{j_1}$, contradiction.

Now, suppose there exists no task $i \in W$ such that $A_i \subseteq V \setminus W$. Then, for $i \in W$, $A_i \cap W \neq \emptyset$. For $j_1 \in W$, let $j_2 \in A_{j_1} \cap W$. Then, $j_1 \notin A_{j_2} \cap W$, else j_1 and j_2 form a cycle. Let $j_3 \in A_{j_2} \cap W$. Consequently, $j_1, j_2 \notin A_{j_3} \cap W$, else j_1 and j_3 , j_2 and j_3 , or j_1, j_2 , and j_3 form a cycle. Repeating

this process, eventually either W has a cycle which contradicts the result in the previous paragraph, or $A_{j_i} \cap W = \emptyset$ for some $j_i \in W$. $\square$

Examples of Procedure FixedLearning

We illustrate the effectiveness of Procedure FixedLearning using two examples. For simplicity, we omit the start and end dummy tasks. We also ignore the resource constraints, i.e., we have no constraint set (8) in the two instances. For $i \in V$, we ignore the direct learning cost, i.e., we set $c_i = 0$. Finally, we let $\bar{y}_i$ be large so that $y_i < \bar{y}_i$, thus learning is not restricted by an upper bound.

Figure EC.3 Initial Status

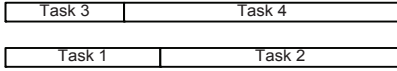
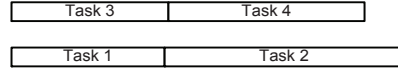


Figure EC.4 After AddLearning



In the first example, we have four tasks in the project, and the precedence constraint set $E^T = \{(1,2), (3,4)\}$. Let $\epsilon > 0$ be a very small value. We have $t_1 = 4 - \epsilon$, $t_2 = 6 + \epsilon$, $t_3 = 3$, $t_4 = 7$, as shown in Figure EC.3. Let the objective function be $\theta C_2 + C_4$, where $\theta \geq 1$. Without learning, we have $C_2 = C_4 = 10$ so that the initial objective value is $H_0 = 10(\theta + 1)$.

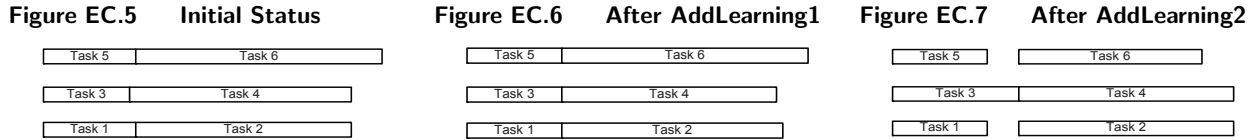
Let $q_{1j} = 0$, for $j = 2, 3, 4$. Thus, task 1 is very different from tasks 2, 3, 4, and thus learning at task 1 is useless. However, learning at task 3 is effective at reducing C_4 . Let $p_4 = t_4 - 2y_3 = 7 - 2y_3$ for $0 \leq y_3 \leq 1$, and $p_4 = t_4 - 2 = 5$ for $y_3 > 1$. After applying Procedure AddLearning, we have $y_3 = 1$ and $C_4 = 9$. In the second round of AddLearning, note that C_2 is still 10, since $C_3 = 4 > S_2 = 4 - \epsilon$ and task 3 cannot be added to the learning set of task 2. Then, Procedure AddLearning stops with a cost of $H_1 = 10\theta + 9$, as shown in Figure EC.4.

The above difficulty that task 3 cannot be added to the learning set of task 2 can be handled by FixedLearning. For the given $y_3 = 1$, we can start task 2 at time 4, later than before, but gain benefit from learning at task 3. Let $p_2 = t_2 - (1.5 + \epsilon)y_3$ for $0 \leq y_3 \leq 1$, and $p_2 = t_2 - (1.5 + \epsilon) = 4.5$ for $y_3 > 1$. With FixedLearning, the objective value is $H_2 = 8.5\theta + 9$.

Thus, we can see that without FixedLearning, the cost reduction by Procedure AddLearning is $H_0 - H_1 = 1$. However, after adding FixedLearning, the cost is reduced further at least by $H_1 - H_2 = 1.5\theta$. As θ increases, FixedLearning saves more cost.

In the above example, we demonstrate the first role of FixedLearning: *adding some tasks to the learning set of some other tasks that cannot be done by AddLearning*. Using a second example, we demonstrate the second role of FixedLearning: *removing some tasks from the learning set of some other tasks that has been added by AddLearning*.

In the second example shown in Figure EC.5, we have six tasks in the project, with nominal processing times $t_1 = t_5 = 3$, $t_2 = 7$, $t_3 = 2.9$, $t_4 = 7.1$, and $t_6 = 8$. The precedence constraints are $\{(1,2), (3,4), (5,6)\}$. Let the objective function be $\theta C_2 + C_4 + \theta C_6$, where $\theta \geq 1$. The task learning



rates are $q_{1j} = 0$ for $j = 2, 3, 4, 5, 6$, and $q_{5j} = 0$, for $j = 1, 2, 3, 4, 6$. Then, there is no learning at tasks 1 and 5. Let $q_{32} = q_{34} = q_{36} = 1$. Since $C_2 = C_4 = 10$ and $C_6 = 11$, we have $H_0 = 21\theta + 10$.

Let $p_4 = t_4 - 2y_3$ for $0 \leq y_3 \leq 0.1$, and $p_4 = t_4 - 0.2 = 6.9$ for $y_3 > 0.1$. After applying AddLearning, $y_3 = 0.1$. Then, $C_4 = 9.9$. Since $S_2 = S_6 = 3 = C_3$, we add task 3 into the learning set of tasks 2 and 6, and $S_2 - C_3 \geq 0$ and $S_6 - C_3 \geq 0$ into Constraint set (4), as shown in Figure EC.6.

Let the learning effect at task 6 be $p_6 = t_6 - 2y_3$ for $0 \leq y_3 \leq 1.1$, and $p_6 = t_6 - 2.2 = 5.8$ for $y_3 > 1.1$. For task 2, let $p_2 = t_2 - \frac{1+\epsilon}{\theta} y_3$ for $0 < y_3 \leq 1.1$, and $p_2 = t_2 - 1.1 \frac{1+\epsilon}{\theta}$ for $y_3 > 1.1$. Then, in the second round of AddLearning, for any $0.1 \leq y_3 \leq 1.1$, we have $C_2 = 2.9 + y_3 + 7 - \frac{1+\epsilon}{\theta} y_3 = 9.9 + (1 - \frac{1+\epsilon}{\theta}) y_3$, $C_4 = 2.9 + y_3 + 6.9 = 9.8 + y_3$, and $C_6 = 2.9 + y_3 + 8 - 2y_3 = 10.9 - y_3$. Hence, $H_1 = \theta C_2 + C_4 + \theta C_6 = 9.9\theta + [\theta - (1 + \epsilon)] y_3 + 9.8 + y_3 + 10.9\theta - \theta y_3 = 20.8\theta + 9.8 - \epsilon y_3$. Then, to minimize the objective value, we let $y_3 = 1.1$. The reduction of the objective value, obtained from applying AddLearning, is then $H_0 - H_1 = 0.2\theta + 0.2 + 1.1\epsilon$ when AddLearning terminates, as shown in Figure EC.7.

In the above process, the learning benefit from task 3 to task 6 is quite high. However, the performance of task 2 is hurt by having task 3 as its learning source, since the knowledge response at task 2, $\frac{1+\epsilon}{\theta}$, is small so that the processing time reduction cannot balance the delay in the starting time. Since AddLearning greedily adds task 3 to the learning sets of both tasks 2 and 6, for further improvement we need FixedLearning to remove task 3 from the learning set of task 2.

After applying FixedLearning, we still have $y_3 = 1.1$, so that $C_4 = 9.8 + y_3$, and $C_6 = 10.9 - y_3$. Now, $C_2 = 10$. Then, $H_2 = \theta C_2 + C_4 + \theta C_6 = 10\theta + 9.8 + y_3 + 10.9\theta - \theta y_3 = 19.8\theta + 10.9$. The cost saving from FixedLearning is at least $H_1 - H_2 = \theta - 1.1 - 1.1\epsilon$. As $\theta \rightarrow \infty$ and $\epsilon \rightarrow 0$, $H_0 - H_1 \approx 0.2\theta$ and $H_1 - H_2 \approx \theta$. As in the first example, as θ increases, FixedLearning saves more cost.

Preliminary Results of Procedure FixedLearning

Lemma EC.1 In a solution developed by Procedure FixedLearning,

1. The completion times of the scheduled tasks do not change when later tasks are scheduled;
2. The completion times of tasks are nondecreasing during schedule construction.

Proof of Lemma EC.1. We first show part (i). If we can show that $C_{l^*}^F > S_j^F$ for any $j \in J$, then the insertion of l^* into J does not change the learning set of any scheduled tasks in J , and hence their completion times will not change. Suppose to the contrary that $C_{l^*}^F \leq S_j^F < C_j^F$ for some $\bar{j} \in J$ and l^* is the first task where this occurs. Then, $S_{l^*}^F < S_{\bar{j}}^F$ and $\bar{j}$ must have been added into J at an earlier step. At that step, let the corresponding scheduled set, unscheduled set, and immediately schedulable set, of tasks be denoted by $\bar{J}$, $\bar{J}_c$, and $\bar{J}'$, respectively. Then, $\bar{j} \in \bar{J}'$. For any i inserted

between $\bar{j}$ and l^* , from our assumption, $C_i^F > S_j^F > S_{l^*}^F$. Therefore, no tasks are added to the learning source of task l^* after the schedule of $\bar{j}$. Hence, the completion time of l^* at the step when task $\bar{j}$ is scheduled is still $C_{l^*}^F$. If $l^* \in \bar{J}'$, then since $C_{l^*}^F < C_{\bar{j}}^F$, i.e., task l^* has a strictly smaller completion time than task $\bar{j}$, task $\bar{j}$ cannot be selected at that step, a contradiction of Step 2. Consequently, $l^* \in \bar{J}_c \setminus \bar{J}'$. Then, there must exist some $v \in \bar{J}'$ so that $(v, l^*) \in E^T$ and $C_v^F < C_{l^*}^F < C_{\bar{j}}^F$. Again, task v has a strictly smaller completion time than task $\bar{j}$ and can be scheduled immediately. Consequently, task $\bar{j}$ cannot be selected at that step, a contradiction. Therefore, $C_{l^*}^F > S_j^F$ for any $j \in J$.

For part (ii), from the definition of $r(j)$ and l^* , we know that in each step Procedure FixedLearning schedules the unscheduled task that can complete earliest. Also, we show above that the completion times of scheduled tasks does not change after scheduling a new task. Thus, we conclude that during scheduling by Procedure FixedLearning, the task completion times are nondecreasing. $\square$

Lemma EC.1 establishes that the insertion of l^* into J does not change the learning set of any scheduled tasks in J , hence their completion times do not change. Thus, one iteration of Procedure FixedLearning from $J = \{1\}$ to V is sufficient to generate the required adjustment of the learning collection and find an improved solution.

Next, we show that Procedure FixedLearning finds an optimal learning collection, i.e., an optimal learning set and thus the smallest completion time for each task, for a given learning time vector $\mathbf{Y}$. For any task $i \in V$, we denote its *minimum* completion time over *all feasible solutions* by $C_i^{\min}(\mathbf{Y})$. Let $\mathcal{F}_i(\mathbf{Y})$ represent the set of feasible solutions that achieve $C_i^{\min}(\mathbf{Y})$. For any feasible solution $F \in \mathcal{F}_i(\mathbf{Y})$, let A_i^F denote the learning set of task i . Finally, for any task $i \in V$, let $A_i^{\min}(\mathbf{Y}) = \bigcup_{F \in \mathcal{F}_i(\mathbf{Y})} A_i^F$. Thus, $A_i^{\min}(\mathbf{Y})$ contains all possible learning sets for task i that enable it to achieve its minimum possible completion time.

We now present several structural results about an optimal solution, for a given $\mathbf{Y}$, of problem (P) . The overall purpose is to show that there exists an optimal solution where, for any $i \in V$, we have $C_i^* = C_i^{\min}(\mathbf{Y})$, i.e., each task achieves its minimum completion time. Observe that a solution which achieves such a configuration, if one exists, is clearly optimal for given $\mathbf{Y}$. In order to show the existence of such an optimal solution in the following Lemma EC.4, we need a definition and two preliminary results, which are presented as Lemmas EC.2 and EC.3 below.

Definition 3 Given $\mathbf{Y}$, we call a sequence of $m \geq 2$ tasks, $j_1, j_2, \dots, j_m \in W \subseteq V$, a mincycle in set W if $j_1 \in A_{j_2}^{\min}(\mathbf{Y})$, $j_2 \in A_{j_3}^{\min}(\mathbf{Y})$, $\dots$, $j_m \in A_{j_1}^{\min}(\mathbf{Y})$.

Lemma EC.2 Given $\mathbf{Y}$, for any task $i \in V$, if $j \in A_i^{\min}(\mathbf{Y})$, then $C_j^{\min}(\mathbf{Y}) < C_i^{\min}(\mathbf{Y})$.

Proof of Lemma EC.2. Since $j \in A_i^{\min}(\mathbf{Y})$, there must exist a feasible schedule $F \in \mathcal{F}_i(\mathbf{Y})$ such that $j \in A_i^F \subseteq A_i^{\min}(\mathbf{Y})$, and $C_i^F = C_i^{\min}(\mathbf{Y})$. Since $j \in A_i^F$, we have $S_i^F \geq C_j^F$. Therefore, $C_j^{\min}(\mathbf{Y}) \leq C_j^F \leq S_i^F < C_i^F = C_i^{\min}(\mathbf{Y})$. $\square$

Lemma EC.2 is intuitive, since if task j is in the learning set of task i in some solution that achieves the minimum completion time of task i , then the minimum completion time of task j must be smaller than that of task i .

Lemma EC.3 Given $\mathbf{Y}$, for any nonempty set $W \subset V$, there must exist a task $i \in W$ such that $A_i^{\min}(\mathbf{Y}) \subseteq V \setminus W$.

Proof of Lemma EC.3. We assume $A_i^{\min}(\mathbf{Y}) \neq \emptyset$, else the result holds trivially. First, we show that any nonempty set $W \subset V$ cannot contain a mincycle. If $|W| < 2$, then W cannot contain a mincycle, by definition. If $|W| \geq 2$, suppose there exists a mincycle of tasks $j_1, j_2, \dots, j_m \in W$ such that $j_1 \in A_{j_2}^{\min}(\mathbf{Y})$, $j_2 \in A_{j_3}^{\min}(\mathbf{Y})$, $\dots$, $j_m \in A_{j_1}^{\min}(\mathbf{Y})$. Then, from Lemma EC.2, $C_{j_1}^{\min}(\mathbf{Y}) < C_{j_2}^{\min}(\mathbf{Y}) < C_{j_3}^{\min}(\mathbf{Y}) < \dots < C_{j_m}^{\min}(\mathbf{Y}) < C_{j_1}^{\min}(\mathbf{Y})$, contradiction.

Now, suppose there exists no task $i \in W$ so that $A_i^{\min}(\mathbf{Y}) \subseteq V \setminus W$. Then, for $i \in W$, $A_i^{\min}(\mathbf{Y}) \cap W \neq \emptyset$. For $j_1 \in W$, let $j_2 \in A_{j_1}^{\min}(\mathbf{Y}) \cap W$. Then, $j_1 \notin A_{j_2}^{\min}(\mathbf{Y}) \cap W$, else j_1 and j_2 form a mincycle. Let $j_3 \in A_{j_2}^{\min}(\mathbf{Y}) \cap W$. Consequently, $j_1, j_2 \notin A_{j_3}^{\min}(\mathbf{Y}) \cap W$, else j_1 and j_3 , j_2 and j_3 , or j_1 , j_2 , and j_3 form a mincycle. Repeating this process, eventually either W has a mincycle which contradicts the result in the previous paragraph, or $A_{j_i}^{\min}(\mathbf{Y}) \cap W = \emptyset$ for some $j_i \in W$. $\square$

For any feasible solution F , we let $W^F(\mathbf{Y}) = \{j \in V | C_j^F > C_j^{\min}(\mathbf{Y})\}$, where C_j^F denotes the completion time of task j in solution F . Then, $W^F(\mathbf{Y})$ represents the set of tasks with completion time in solution F that are not minimum for given $\mathbf{Y}$.

Lemma EC.4 Given $\mathbf{Y}$, there exists an optimal solution where $C_i^* = C_i^{\min}(\mathbf{Y})$, for $i \in V$.

Proof of Lemma EC.4. For any feasible solution F , if $W^F(\mathbf{Y}) = \emptyset$, then $\forall i \in V$, $C_i^F = C_i^{\min}(\mathbf{Y})$, which completes the proof. If $W^F(\mathbf{Y}) \neq \emptyset$, then from the definition of $W^F(\mathbf{Y})$ and Lemma EC.3, there exists a task $l \in W^F(\mathbf{Y})$, such that $C_l^F > C_l^{\min}(\mathbf{Y})$ and $A_l^{\min}(\mathbf{Y}) \subseteq V \setminus W^F(\mathbf{Y})$.

Recall that $\mathcal{F}_i(\mathbf{Y})$ represents the set of feasible solutions that realize $C_i^{\min}(\mathbf{Y})$. For any feasible solution $F' \in \mathcal{F}_l(\mathbf{Y})$, we use $A_l^{F'}$ to denote the learning set of task l in solution F' , $S_l^{F'}$ to denote its start time in F' , and $p_l^{F'}$ to denote its duration in F' . From the definition of $A_l^{\min}(\mathbf{Y})$, we have $A_l^{F'} \subseteq A_l^{\min}(\mathbf{Y}) \subseteq V \setminus W^F(\mathbf{Y})$.

Now, starting with solution F , we construct another feasible schedule F'' by setting $S_l^{F''} = S_l^F$ and $C_i^{F''} = C_i^F$ for $i \in V \setminus \{l\}$, and $S_l^{F''} = \max_{i \in A_l^{F'}} \{C_i^F\}$, i.e., the maximum completion time of the tasks in the learning set $A_l^{F'}$ under the solution F . Since task l starts after all tasks in $A_l^{F'}$ have been completed in F'' , we have $A_l^{F''} \supseteq A_l^{F'}$. This implies $p_l^{F''} \leq p_l^{F'}$, since l has a learning set in F'' larger than or equal to that in F' , for given $\mathbf{Y}$.

As shown above, $A_l^{F'} \subseteq A_l^{\min}(\mathbf{Y}) \subseteq V \setminus W^F(\mathbf{Y})$. Then, from the definition of $W^F(\mathbf{Y})$, $C_i^F = C_i^{\min}(\mathbf{Y})$, for any $i \in A_l^{F'}$. Thus, $S_l^{F'} \geq \max_{i \in A_l^{F'}} \{C_i^{F'}\} \geq \max_{i \in A_l^{F'}} \{C_i^{\min}(\mathbf{Y})\} = \max_{i \in A_l^{F'}} \{C_i^F\} = S_l^{F''}$. Also, we have established above that $p_l^{F''} \leq p_l^{F'}$, and y_l is given. Therefore, $C_l^{F''} = S_l^{F''} + p_l^{F''} +$

$y_l \leq S_l^{F'} + p_l^{F'} + y_l = C_l^{min}(\mathbf{Y})$, from the definition of $\mathcal{F}_l(\mathbf{Y})$ and the fact that $F' \in \mathcal{F}_l(\mathbf{Y})$. As a result, $C_l^{F''} = C_l^{min}(\mathbf{Y})$. From the definition of l , we have $C_l^F > C_l^{min}(\mathbf{Y})$. Thus, task l completes earlier in solution F'' than in solution F , whereas the start and completion times of the other tasks remain the same as in F . Since solution F is optimal by assumption, solution F'' is optimal as well.

While $W^F(\mathbf{Y}) \neq \emptyset$, we repeat this transformation and remove one element from $W^F(\mathbf{Y}) \neq \emptyset$ until the lemma holds. $\square$

From Lemma EC.4, for the optimal solution where each task achieves its minimal completion time, the “cycle” in Definition 2 turns out to be a “mincycle” in Definition 3. By making use of the result in Lemma EC.4, Procedure FixedLearning finds the minimal completion time for each task sequentially.

Lemma EC.5. Given $\mathbf{Y}$ and a partial network $G(J, E(J))$, where $1 \in J$ and $J \subset V \setminus \{n\}$. Let F denote a schedule corresponding to this partial network with $C_j^F = C_j^{min}(\mathbf{Y})$, for $j \in J$. Let task l^* be scheduled to start at $r(l^*)$ so that $C_{l^*}^F = T^F(l^*, r(l^*))$. Then, in the extended partial network $G(J \cup \{l^*\}, E(J \cup \{l^*\}))$, we have $C_j^F = C_j^{min}(\mathbf{Y})$, for $j \in J \cup \{l^*\}$.

Proof of Lemma EC.5. We first show that $\min_{i \in J_c} \{C_i^{min}(\mathbf{Y})\} = \min_{i \in J'} \{C_i^{min}(\mathbf{Y})\}$. This is trivial if $J_c \setminus J' = \emptyset$, hence we let $J_c \setminus J' \neq \emptyset$. Observe that for each $i \in J_c \setminus J'$, there exists at least one $j_i \in J_c$ where $(j_i, i) \in E$, else $i \in J'$. Let $m = \arg \min_{i \in J_c \setminus J'} \{C_i^{min}(\mathbf{Y})\}$. Then, $j_m \in J'$, since otherwise, from $C_{j_m}^{min}(\mathbf{Y}) < C_m^{min}(\mathbf{Y})$ which is implied by Lemma EC.2, we have $j_m = \arg \min_{i \in J_c \setminus J'} \{C_i^{min}(\mathbf{Y})\}$, a contradiction. Now, since $j_m \in J'$, from $C_{j_m}^{min}(\mathbf{Y}) < C_m^{min}(\mathbf{Y})$, we have $\min_{i \in J_c} \{C_i^{min}(\mathbf{Y})\} = \min_{i \in J'} \{C_i^{min}(\mathbf{Y})\}$.

Next, we show that $C_{l^*}^{min}(\mathbf{Y}) = \min_{i \in J_c} \{C_i^{min}(\mathbf{Y})\}$. Suppose to the contrary that $C_{l^*}^{min}(\mathbf{Y}) > \min_{i \in J_c} \{C_i^{min}(\mathbf{Y})\}$. It follows that $C_{l^*}^{min}(\mathbf{Y}) > \min_{i \in J'} \{C_i^{min}(\mathbf{Y})\}$. Let $h \in J'$ such that $C_h^{min}(\mathbf{Y}) = \min_{i \in J'} \{C_i^{min}(\mathbf{Y})\} < C_{l^*}^{min}(\mathbf{Y})$. Since $l^* \in J'$, we have $h \in J' \setminus \{l^*\}$. Moreover, recall that $l^* = \arg \min_{j \in J'} \{T^F(j, r(j))\}$. As a result, we have $C_h^{min}(\mathbf{Y}) < C_{l^*}^{min}(\mathbf{Y}) \leq T^F(l^*, r(l^*)) \leq T^F(h, r(h))$. There are two cases.

First, suppose $J_c \cap A_h^{min}(\mathbf{Y}) = \emptyset$. Then, all the learning tasks that are necessary to achieve $C_h^{min}(\mathbf{Y})$ are in J . Also, the completion times of the tasks in $R_h \subseteq J$ define all the possible starting times of task h , and by assumption $C_j^F = C_j^{min}(\mathbf{Y})$ for $j \in J$. Then, we have $T^F(h, r(h)) = C_h^{min}(\mathbf{Y})$, which contradicts the fact that $C_h^{min}(\mathbf{Y}) < T^F(h, r(h))$.

Second, suppose $J_c \cap A_h^{min}(\mathbf{Y}) \neq \emptyset$. Then, from Lemma EC.2, for any task $j \in J_c \cap A_h^{min}(\mathbf{Y})$, we have $C_j^{min}(\mathbf{Y}) < C_h^{min}(\mathbf{Y})$, which contradicts the fact that $C_h^{min}(\mathbf{Y}) = \min_{i \in J_c} \{C_i^{min}(\mathbf{Y})\} = \min_{i \in J'} \{C_i^{min}(\mathbf{Y})\}$.

From the above two cases, we conclude that $C_{l^*}^{min}(\mathbf{Y}) = \min_{i \in J_c} \{C_i^{min}(\mathbf{Y})\}$.

Now, since $C_{l^*}^{min}(\mathbf{Y}) = \min_{i \in J_c} \{C_i^{min}(\mathbf{Y})\}$, we have $C_i^{min}(\mathbf{Y}) \geq C_{l^*}^{min}(\mathbf{Y})$, for $i \in J_c$. Then, from the contrapositive of Lemma EC.2, we have $J_c \cap A_{l^*}^{min}(\mathbf{Y}) = \emptyset$. Also, since by assumption $C_j^F = C_j^{min}(\mathbf{Y})$ for any $j \in J$, all tasks in J are already scheduled to finish at their smallest completion time. Thus,

for task l^* , we cannot find a completion time smaller than $T^F(l^*, r(l^*))$, and hence $C_{l^*}^F = C_{l^*}^{min}(\mathbf{Y}) = T^F(l^*, r(l^*))$.

Moreover, Lemma EC.1 ensures that after adding l^* to J , the completion times of tasks in J do not change, i.e., $C_j^F = C_j^{min}(\mathbf{Y})$, for $j \in J$. $\square$

Lemma EC.5 establishes that we can find a task l^* for which the learning set required to achieve its minimum completion time has been scheduled earlier in J , and hence task l^* achieves its minimum completion time as well.

Proof of Theorem 3. The feasibility and optimality of Procedure FixedLearning follow from Lemmas EC.4 and EC.5, respectively. For the running time, Step 2 requires $O(n)$ iterations until $J = V$. Within each iteration, for each $j \in J'$, there are $O(n)$ computations for $r(j)$ from R_j , and $|J'| = O(n)$. Consequently, the overall time complexity of FixedLearning is $O(n^3)$. $\square$

Proof of Lemma 5. If $q_{ij} \cdot \bar{y}_i - (\bar{C}_i - \underline{S}_j)M_{ij} > 0$, then $q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} \geq q_{ij} \cdot \bar{y}_i - (\bar{C}_i - \underline{S}_j)M_{ij} > 0$ and from the original Constraints (20) - (22), we have $x_{ij}^+ > 0$ and $x_{ij}^- = 0$. Then, $x_{ij}^- \leq \bar{x}_{ij}^- = 0$. Alternatively, if $q_{ij} \cdot \bar{y}_i - (\bar{C}_i - \underline{S}_j)M_{ij} \leq 0$, then $\bar{x}_{ij}^- = -(q_{ij} \cdot \bar{y}_i - (\bar{C}_i - \underline{S}_j)M_{ij})$. In this case, the optimal solution of problem (PR) can be evaluated in two subcases:

1. If $q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} < 0$, then $x_{ij}^+ = 0$ and $x_{ij}^- = (C_i - S_j)M_{ij} - q_{ij} \cdot \bar{y}_i$. Consequently, $x_{ij}^- \leq \bar{x}_{ij}^-$.
2. If $q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} \geq 0$, then $x_{ij}^+ \geq 0$ and $x_{ij}^- = 0 \leq \bar{x}_{ij}^-$.

For $\bar{u}_{ij}^-$, if $\delta_{ij} - (\bar{C}_i - \underline{S}_j)W_{ij} > 0$, then $\delta_{ij} - (C_i - S_j)W_{ij} \geq \delta_{ij} - (\bar{C}_i - \underline{S}_j)W_{ij} > 0$ and from the original Constraints (25) - (27), we have $u_{ij}^+ > 0$ and $u_{ij}^- = 0$. Then, $u_{ij}^- \leq \bar{u}_{ij}^- = 0$. Alternatively, if $\delta_{ij} - (\bar{C}_i - \underline{S}_j)W_{ij} \leq 0$, then $\bar{u}_{ij}^- = -(\delta_{ij} - (\bar{C}_i - \underline{S}_j)W_{ij})$. In this case, the optimal solution of problem (PR) can be evaluated in two subcases:

1. If $\delta_{ij} - (C_i - S_j)W_{ij} < 0$, then $u_{ij}^+ = 0$ and $u_{ij}^- = (C_i - S_j)W_{ij} - \delta_{ij}$. Consequently, $u_{ij}^- \leq \bar{u}_{ij}^-$.
2. If $\delta_{ij} - (C_i - S_j)W_{ij} \geq 0$, then $u_{ij}^+ \geq 0$ and $u_{ij}^- = 0 \leq \bar{u}_{ij}^-$. $\square$

Proof of Theorem 4. Let $\tilde{\mathbf{M}} = (\tilde{M}_{ij})$, where $\tilde{M}_{ij} = M_{ij}$ holds for all tasks $i, j \in V$ and $i \neq j$, except that $\tilde{M}_{lr} > M_{lr}$ for some $l, r \in V$ where $l \neq r$. We first show $Obj^*(PR(\tilde{\mathbf{M}}, \mathbf{W})) \geq Obj^*(PR(\mathbf{M}, \mathbf{W}))$.

Denote the optimal solution of problem $(PR(\tilde{\mathbf{M}}, \mathbf{W}))$ by $(\tilde{\mathbf{S}}^*, \tilde{\mathbf{Y}}^*, \tilde{T}^*, \tilde{\mathbf{X}}^{+*}, \tilde{\mathbf{X}}^{-*}, \tilde{\mathbf{U}}^{+*}, \tilde{\mathbf{U}}^{-*})$. If $\tilde{C}_l^* \leq \tilde{S}_r^*$, then Constraint (21) for l and r is redundant, and the optimal solution is independent of $\tilde{M}_{lr}$ and M_{lr} . Then, $Obj^*(PR(\tilde{\mathbf{M}}, \mathbf{W})) = Obj^*(PR(\mathbf{M}, \mathbf{W}))$. Alternatively, if $\tilde{C}_l^* > \tilde{S}_r^*$, then since $\tilde{M}_{lr} > M_{lr}$, we have $\tilde{x}_{lr}^{+*} - \tilde{x}_{lr}^{-*} \leq q_{lr} \cdot \bar{y}_l - (\tilde{C}_l^* - \tilde{S}_r^*)\tilde{M}_{lr} < q_{lr} \cdot \bar{y}_l - (\tilde{C}_l^* - \tilde{S}_r^*)M_{lr}$. Thus, $(\tilde{\mathbf{S}}^*, \tilde{\mathbf{Y}}^*, \tilde{T}^*, \tilde{\mathbf{X}}^{+*}, \tilde{\mathbf{X}}^{-*}, \tilde{\mathbf{U}}^{+*}, \tilde{\mathbf{U}}^{-*})$ is a feasible solution of $PR(\mathbf{M}, \mathbf{W})$, and hence $Obj^*(PR(\tilde{\mathbf{M}}, \mathbf{W})) \geq Obj^*(PR(\mathbf{M}, \mathbf{W}))$.

Likewise, let $\tilde{\mathbf{W}} = (\tilde{W}_{ij})$, where $\tilde{W}_{ij} = W_{ij}$ holds for all tasks $i, j \in V$ and $i \neq j$, except that $\tilde{W}_{lr} > W_{lr}$ for some $l, r \in V$ where $l \neq r$. Denote the optimal solution of problem $(PR(\mathbf{M}, \tilde{\mathbf{W}}))$ by $(\hat{\mathbf{S}}^*, \hat{\mathbf{Y}}^*, \hat{T}^*, \hat{\mathbf{X}}^{+*}, \hat{\mathbf{X}}^{-*}, \hat{\mathbf{U}}^{+*}, \hat{\mathbf{U}}^{-*})$. If $\hat{C}_l^* \leq \hat{S}_r^*$, then Constraint (26) for l and r is redundant, and the optimal solution is independent of $\tilde{W}_{lr}$ and W_{lr} . Then, $Obj^*(PR(\hat{\mathbf{M}}, \mathbf{W})) = Obj^*(PR(\mathbf{M}, \mathbf{W}))$.

Alternatively, if $\hat{C}_l^* > \hat{S}_r^*$, then since $\tilde{W}_{lr} > W_{lr}$, we have $\hat{u}_{lr}^{+*} - \hat{u}_{lr}^{-*} \leq \delta_{lr} - (\hat{C}_l^* - \hat{S}_r^*)\tilde{W}_{lr} < \delta_{lr} - (\hat{C}_l^* - \hat{S}_r^*)W_{lr}$. Thus, $(\hat{\mathbf{S}}^*, \hat{\mathbf{Y}}^*, \hat{T}^*, \hat{\mathbf{X}}^{+*}, \hat{\mathbf{X}}^{-*}, \hat{\mathbf{X}}^{-*}, \hat{\mathbf{U}}^{+*}, \hat{\mathbf{U}}^{-*})$ is a feasible solution of $PR(\mathbf{M}, \mathbf{W})$, and hence $Obj^*(PR(\mathbf{M}, \tilde{\mathbf{W}})) \geq Obj^*(PR(\mathbf{M}, \mathbf{W}))$.

When $\mathbf{M} \rightarrow +\infty$ and $\mathbf{W} \rightarrow +\infty$, for any two tasks i and j , if $C_i \leq S_j$, Constraints (21) and (26) are redundant, and $x_{ij}^{+*} = q_{ij}y_i$, $u_{ij}^{+*} = \delta_{ij}$; alternatively, if $C_i > S_j$, then since $M_{ij} \rightarrow +\infty$, we have $q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} \rightarrow -\infty$ and $x_{ij}^{+*} = 0$; Since $W_{ij} \rightarrow +\infty$, we have $\delta_{ij} - (C_i - S_j)W_{ij} \rightarrow -\infty$ and $u_{ij}^{+*} = 0$. Thus, in both cases, $PR(+\infty)$ is equivalent to (P) , i.e., $Obj^*(PR(+\infty)) = Obj^*(P)$. $\square$

Proof of Theorem 5. In problem (PR1), the value of $M_{ij} \geq 0$ affects only Constraints (21). There are two cases.

1. Suppose $M_{ij} \leq \frac{q_{ij} \cdot \bar{y}_i}{\bar{C}_i - \underline{S}_j}$. Then, $q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij} \geq q_{ij} \cdot \bar{y}_i - (\bar{C}_i - \underline{S}_j)M_{ij} \geq 0$. Consequently, $\bar{x}_{ij}^- = 0$, $x_{ij}^- = 0$, and Constraint (38) becomes trivial. Moreover, Constraint (21) simplifies to $x_{ij}^+ \leq q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij}$, where the right hand side value becomes smaller as M_{ij} increases. As a result, the feasible region of (PR1) becomes smaller, and the relaxation becomes tighter.
2. Suppose $M_{ij} \geq \frac{q_{ij} \cdot \bar{y}_i}{\bar{C}_i - \underline{S}_j}$. Then, $q_{ij} \cdot \bar{y}_i - (\bar{C}_i - \underline{S}_j)M_{ij} \leq 0$ and $\bar{x}_{ij}^- = (\bar{C}_i - \underline{S}_j)M_{ij} - q_{ij} \cdot \bar{y}_i \geq 0$. Combining Constraints (21) and (38) yields

$$x_{ij}^+ \leq \min \left\{ x_{ij}^- + q_{ij} \cdot \bar{y}_i - (C_i - S_j)M_{ij}, \quad q_{ij} \cdot \bar{y}_i - q_{ij} \cdot \bar{y}_i \frac{x_{ij}^-}{\bar{x}_{ij}^-} \right\}. \quad (\text{EC.8})$$

For given S_j , C_i , W_{ij} and M_{ij} , to minimize the objective function, the solution of the linear program (PR1) makes x_{ij}^+ as large as possible, by adjusting the value of x_{ij}^- . In particular, the right hand side value of (EC.8) is maximized when its two terms are equal, which implies $\frac{x_{ij}^-}{\bar{x}_{ij}^-} = \frac{C_i - S_j}{\bar{C}_i - \underline{S}_j}$. As a result, constraint (EC.8) is simplified to (44), which is independent of M_{ij} .

Likewise, the value of $W_{ij} \geq 0$ affects only constraints (26). There are two cases.

1. Suppose $W_{ij} \leq \frac{\delta_{ij}}{\bar{C}_i - \underline{S}_j}$. Then, $\delta_{ij} - (C_i - S_j)W_{ij} \geq \delta_{ij} - (\bar{C}_i - \underline{S}_j)W_{ij} \geq 0$. Consequently, $\bar{u}_{ij}^- = 0$, $u_{ij}^- = 0$, and Constraint (39) becomes trivial. Moreover, Constraint (26) simplifies to $u_{ij}^+ \leq \delta_{ij} - (C_i - S_j)W_{ij}$, where the right hand side value becomes smaller as W_{ij} increases. As a result, the feasible region of (PR1) becomes smaller, and the relaxation becomes tighter.
2. Suppose $W_{ij} \geq \frac{\delta_{ij}}{\bar{C}_i - \underline{S}_j}$. Then, $\delta_{ij} - (\bar{C}_i - \underline{S}_j)W_{ij} \leq 0$ and $\bar{u}_{ij}^- = (\bar{C}_i - \underline{S}_j)W_{ij} - \delta_{ij} \geq 0$. Combining Constraints (26) and (39) yields

$$u_{ij}^+ \leq \min \left\{ u_{ij}^- + \delta_{ij} - (C_i - S_j)W_{ij}, \quad \delta_{ij} - \delta_{ij} \frac{u_{ij}^-}{\bar{u}_{ij}^-} \right\}. \quad (\text{EC.9})$$

For given S_j , C_i , M_{ij} and W_{ij} , to minimize the objective function, the solution of the linear program (PR1) makes u_{ij}^+ as large as possible, by adjusting the value of u_{ij}^- . In particular, the right hand side value of (EC.9) is maximized when its two terms are equal, which implies $\frac{u_{ij}^-}{\bar{u}_{ij}^-} = \frac{C_i - S_j}{\bar{C}_i - \underline{S}_j}$. As a result, constraint (EC.9) is simplified to (45), which is independent of W_{ij} . $\square$

Establishment of $\overline{C}_i$ and $\underline{S}_i$, Elimination Criteria, and Branch and Bound Algorithm

We now establish elimination criteria that can be used to eliminate dominated solutions. To build such criteria, we consider a project scheduling problem with $G = (V, E)$, $p_i = t_i + \bar{y}_i$, for any $i \in V$, i.e., the maximal processing time of each task. Then, we use the critical path method (CPM) to compute the *early finish time*, which we denote by $\overline{EF}_i$, for each task $i \in V$. We have the following result about $\overline{EF}_i$.

Lemma EC.6. There exists an optimal schedule such that the optimal completion time C_i of task i is no larger than $\overline{EF}_i$, i.e., $C_i^* \leq \overline{EF}_i$, $\forall i \in V$.

Proof of Lemma EC.6. The worst possible schedule is obtained if task i used its maximum learning time $\bar{y}_i$, $i = 1, \dots, n$, but no reduction in processing time results. Then, by construction, $C_i^* \leq \overline{EF}_i$, $i = 1, \dots, n$. $\square$

For each task $i \in V$, we refer to $A'_i = A_i \cup \{j \in V \mid [i, j] \in \Gamma \text{ or } [j, i] \in \Gamma\}$ as *all potential learning source sets* of task i . Recall that we require $i < j$ for $[i, j] \in \Gamma$. Then, A'_i only excludes the tasks that must start after the completion of task i , and thus cannot be learning sources of task i . Given a partial collection of learning sets $\mathbf{A}$ and the collection of all potential learning sets $\mathbf{A}' \supseteq \mathbf{A}$, we use $\underline{ES}_i$ to denote the earliest start time of task i . Formally, $\underline{ES}_i$ is the solution of the following linear program (PS_i):

$$\begin{aligned} (PS_i) \quad & \min_{\mathbf{S}, \mathbf{Y}} S_i \\ \text{s.t.} \quad & \text{Constraints (4), (7) and (8),} \\ & C_i - S_i = t_i - f_i\left(\sum_{j \in A'_i} q_{ji} y_j\right) - g_i\left(\sum_{j \in A'_i} \delta_{ji}\right) + y_i, \quad i \in V. \end{aligned} \quad (\text{EC.10})$$

In Constraints (EC.10), all possible knowledge is available for each task, since $\mathbf{A} \subseteq \mathbf{A}'$. Thus, $\underline{ES}_i \leq S_i^*$. Similarly, the earliest completion time of task i , $\underline{EF}_i$, is the optimal solution of the following problem (PC_i):

$$\begin{aligned} (PC_i) \quad & \min_{\mathbf{S}, \mathbf{Y}} C_i \\ \text{s.t.} \quad & \text{Constraints (4), (7), (8) and (EC.10).} \end{aligned}$$

Again, since we use all the possible knowledge for each task, we have $\underline{EF}_i \leq C_i^*$.

Using the values of $\overline{EF}_i$, $\underline{ES}_i$, and $\underline{EF}_i$, we develop the following two results as elimination rules. Let $\underline{p}_i$ be the minimum duration of task i , after applying all the accumulated knowledge.

Lemma EC.7 For $[i, j] \in \Gamma$, if $\overline{EF}_i - \bar{y}_i < \underline{EF}_j + \underline{p}_i$, then there exists an optimal schedule where $S_i^* < C_j^*$.

Proof of Lemma EC.7. We first show that $C_i^* - y_i^* \leq \overline{EF}_i - \bar{y}_i$. There are two cases.

Case 1: The learning set of task i has not been enlarged. Suppose $S_i^* = \max_{(l, i) \in E} \{C_l^*\}$. Then, from the definition of $\overline{EF}_i$ and Lemma EC.6, we have $S_i^* = \max_{(l, i) \in E} \{C_l^*\} \leq \max_{(l, i) \in E} \{\overline{EF}_l\}$,

and since $t_i \geq p_i^*$ for $i \in V$ we have $S_i^* + p_i^* \leq \max_{(l,i) \in E} \{\overline{EF}_l\} + t_i$. From $C_i^* - y_i^* = S_i^* + p_i^*$ and $\overline{EF}_i - \bar{y}_i = \max_{(l,i) \in E} \{\overline{EF}_l\} + t_i$, we have $C_i^* - y_i^* \leq \overline{EF}_i - \bar{y}_i$. Alternatively, if $S_i^* > \max_{(l,i) \in E} \{C_l^*\}$, we can construct a solution F so that $y_j^F = y_j^*$ for any $j \in V$, $S_j^F = S_j^*$ for $j \in V \setminus \{i\}$, and $S_i^F = \max_{(l,i) \in E} \{C_l^*\} = \max_{(l,i) \in E} \{C_l^F\}$. Since task i finishes earlier and no other task completion times have changed, solution F is no worse than the original one and is therefore optimal. For this optimal solution F , we have $C_i^F - y_i^F \leq \overline{EF}_i - \bar{y}_i$.

Case 2: The learning set of task i has been enlarged. Some new precedence tasks have been added to task i in the optimal schedule. If $S_i^* + p_i^* \leq \max_{(l,i) \in E} \{\overline{EF}_l\} + t_i$, then from $C_i^* - y_i^* = S_i^* + p_i^*$ and $\overline{EF}_i - \bar{y}_i = \max_{(l,i) \in E} \{\overline{EF}_l\} + t_i$, we have $C_i^* - y_i^* \leq \overline{EF}_i - \bar{y}_i$. Suppose to the contrary that $S_i^* + p_i^* > \max_{(l,i) \in E} \{\overline{EF}_l\} + t_i$. Then, we can construct a solution F by eliminating these added precedence relationships of task i , and setting $y_j^F = y_j^*$ for any $j \in V$, $S_j^F = S_j^*$ for $j \in V \setminus \{i\}$, and $S_i^F = \max_{(l,i) \in E} \{\overline{EF}_l\} < S_i^* + p_i^* - t_i \leq S_i^*$. By doing this, the learning sources of task i shrink, hence the processing time of i may increase, but nonetheless $p_i^F \leq t_i$. Then, the completion time of task i under solution F satisfies $C_i^F - y_i^F = S_i^F + p_i^F \leq \max_{(l,i) \in E} \{\overline{EF}_l\} + t_i < S_i^* + p_i^* = C_i^* - y_i^*$, hence $C_i^F < C_i^*$. Since task i finishes earlier and no other task completion times are changed, solution F is no worse than the original one and is therefore optimal. For this optimal solution F , since $S_i^F + p_i^F \leq \max_{(l,i) \in E} \{\overline{EF}_l\} + t_i$, we have $C_i^F - y_i^F \leq \overline{EF}_i - \bar{y}_i$.

Cases 1 and 2 together establish that $C_i^* - y_i^* \leq \overline{EF}_i - \bar{y}_i$. Now suppose, in contradiction to the lemma statement, that $S_i^* \geq C_j^*$. If $\overline{EF}_i - \bar{y}_i < \underline{EF}_j + \underline{p}_i$, then $C_i^* = S_i^* + p_i^* + y_i^* \geq C_j^* + p_i^* + y_i^* \geq \underline{EF}_j + \underline{p}_i + y_i^* > \overline{EF}_i - \bar{y}_i + y_i^*$, where the second inequality follows from $C_j^* \geq \underline{EF}_j$ and $p_i^* \geq \underline{p}_i$. Then, $C_i^* - y_i^* > \overline{EF}_i - \bar{y}_i$, a contradiction. $\square$

Lemma EC.7 provides a condition under which there exists an optimal solution where one task starts before the completion of another task, and hence never uses knowledge from the other task. Further, we have the following lemma about the opposite situation.

Lemma EC.8. *For $[i, j] \in \Gamma$, if $\overline{EF}_j \leq \underline{ES}_i$, then there exists an optimal schedule where $C_j^* \leq S_i^*$.*

Proof of Lemma EC.8. From Lemma EC.6, we have $C_j^* \leq \overline{EF}_j$. Also, from the definition of $\underline{ES}_i$, we know that $S_i^* \geq \underline{ES}_i$. Thus, if $\overline{EF}_j \leq \underline{ES}_i$, we have $C_j^* \leq \overline{EF}_j \leq \underline{ES}_i \leq S_i^*$. $\square$

These two results are effective in reducing the search space in the branch-and-bound algorithm that is now described.

Consider a partially solved instance. Let $v \geq 0$ be a node of the branching tree, where $v = 0$ is the root node. Let $l(v) = 0$ denote that node v has not been explored; $l(v) = -1$ denote that node v cannot contain an optimal solution; and $l(v) = 1$ denote that this node can contain an optimal solution. Let $r(v)$, for $v > 0$, be the parent node of v (i.e., one level higher than v); $s(v)$, for $v \geq 0$, be the set of immediate successor nodes of v on the branching tree (i.e., one level lower than v); and $c(v)$, for $v \geq 0$, be the number of times node v has been visited. Finally, let Γ_v denote the set

of precedence relationships that are given or implied by the partial schedule information at node v , and let Γ_v^T denote the extended set of precedence relationships implied by transitive relations.

Branch-and-Bound Algorithm

We apply our heuristic and lower bounds to identify optimal solutions to problem (P) . Recall that the extended precedence constraint set E^T includes all transitive precedence relations. To solve problem (P) , we need to determine the optimal order of the start times and completion times of all tasks and find the optimal learning collection $\mathbf{A}$. Consider again the example in Figure EC.2. Recall from Equation (9) that the corresponding Γ of the project network is $\{[2, 3], [2, 6], [4, 6], [5, 6]\}$. Then, to solve the problem, we only need to determine the optimal relative timing of task pairs 2 and 3, 2 and 6, 4 and 6, and 5 and 6. Each pair of tasks has the three possible choices for branching. When the order for any one of the task pairs is fixed, the initial learning collection can be updated, and the corresponding upper and lower bounds can be calculated based on the results in Sections 5.1 and 5.2. For example, if we consider a node v that sets task 3 to start after task 2 is completed, the updated initial learning collection for node v will be $E_v^T = E^T \cup \{(2, 3)\}$. Then, the upper and lower bounds of node v can be computed by fixing E_v^T . The details of the branch and bound algorithm are as follows:

0. Let $v = 0$, $\Gamma_v = E$ and $\Gamma_v^T = E^T$. Calculate $\overline{EF}_i$ for each task $i \in V$ by solving the project scheduling problem with data $G = (V, E)$, and $p_i = t_i + \overline{y}_i$, for $i \in V$. For notational convenience, let $\overline{\mathbf{EF}} = (\overline{EF}_i)$, which is a row vector. Let $UB^* = H(\overline{\mathbf{EF}}, \mathbf{0})$ and $CN = \overline{EF}_n$, which records the makespan of the project. Let $c(0) = 0$ and $l(0) = 0$.
1. If $c(v) = 0$, then go to Step 2.
 If $c(v) = 1$, let $c(v) = c(v) + 1$. In $s(v)$, there must be two nodes, denoted by i and j , where $l(i) = l(j) = 0$. Arbitrarily choose one, say i , let $v = i$, and go to Step 2.
 If $c(v) = 2$, let $c(v) = c(v) + 1$. In $s(v)$, there must be a node, denoted by i , where $l(i) = 0$. Let $v = i$, and go to Step 2.
 If $c(v) = 3$ and $v > 0$, then let $v = r(v)$ and go to Step 1; otherwise, stop.
2. Let $\Gamma = \{[i, j] | (i, j) \notin \Gamma_v^T, (j, i) \notin \Gamma_v^T, (i || j) \notin \Gamma_v^T, i < j \in V\}$. Set $A_{v,i} = \{j \in V | (j, i) \in \Gamma_v^T\}$ and $A'_{v,i} = A_{v,i} \cup \{j \in V | [j, i] \in \Gamma \text{ or } [i, j] \in \Gamma\}$ for each task $i \in V$. Let $\mathbf{A}_v$ and $\mathbf{A}'_v$ denote the learning collections of $A_{v,i}$ and $A'_{v,i}$, respectively.
 - a. Calculate $\underline{ES}_i$ from (PS_i) and $\underline{EF}_i$ from (PC_i) , for $i \in V$.
 - b. Using Algorithm DI, given the initial learning source sets $\mathbf{A}_v$, calculate the upper bound UB_v . Let $UB^* = \min\{UB^*, UB_v\}$.
 - c. Consider a project scheduling problem with $G = (V, \Gamma_v)$, where

$$p_i = \begin{cases} t_i, & \text{if } A'_{v,i} = \{1\}, \\ \underline{p}_i, & \text{otherwise,} \end{cases} \quad \text{for } i \in V, \quad (\text{EC.11})$$

- and project deadline CN . Use CPM to compute ES_i , the earliest starting time, and LF_i , the latest finish time, for $i \in V$. If the current problem is infeasible, let $l(v) = -1$, $v = r(v)$ and go to Step 1. Otherwise, let $\underline{S}_i = \max\{ES_i, \underline{ES}_i\}$ and $\overline{C}_i = \min\{LF_i, \overline{EF}_i\}$.
- d. Compute the lower bound LB_v by solving problem (PR2). If $LB_v \geq UB^*$, then let $l(v) = -1$, $v = r(v)$ and go to Step 1. Otherwise, let $l(v) = 1$, and go to Step 3.
3. If $\Gamma = \emptyset$, let $v = r(v)$ and go to Step 1; otherwise, let $i^* = \min_{i,j \in V} \{[i, j] \in \Gamma\}$ and $j^* = \min_{j \in V} \{[i^*, j] \in \Gamma\}$. Let u_i , for $i = 1, 2, 3$, denote the smallest three available positive values that have not been used for branching nodes. Let $s(v) = \{u_1, u_2, u_3\}$, and $r(u_1) = r(u_2) = r(u_3) = v$. Let $\Gamma_{u_1} = \Gamma_v \cup \{(i^*, j^*)\}$, $\Gamma_{u_2} = \Gamma_v \cup \{(j^*, i^*)\}$ and $\Gamma_{u_3} = \Gamma_v \cup \{(i^* \parallel j^*)\}$.
- a. Let $l(u_i) = c(u_i) = 0$, for $i = 1, 2, 3$.
- b. Determine if u_i can be eliminated, and set $l(u_i)$ accordingly, for $i = 1, 2, 3$.
- c. Eliminate branches, as follows:
- i. If $\overline{EF}_{i^*} - \overline{y}_{i^*} < \underline{EF}_{j^*} + \underline{p}_{j^*}$, then let $l(u_2) = -1$, where (j^*, i^*) is eliminated.
 - ii. If $\overline{EF}_{j^*} \leq \underline{ES}_{i^*}$, then let $l(u_1) = l(u_3) = -1$, where (i^*, j^*) and $(i^* \parallel j^*)$ are eliminated and only (j^*, i^*) will be explored.
- d. Construct Γ_j^T from Γ_j by adding any necessary transitive precedence relationships, for $j = u_i$ if $l(u_i) \neq -1$, for $i = 1, 2, 3$.
- e. Let $c(v) = -l(u_1) - l(u_2) - l(u_3) + 1$.
- f. Let $i \in s(v)$ with $l(i) = 0$. Let $v = i$, and go to Step 1.

We provide some explanation of the algorithm. Since each pair $[i, j]$ generates at most three cases (i, j) , (j, i) , and $(i \parallel j)$, a node v is visited at most three times, using depth-first-search. In Step 0, we start from the root node $v = 0$, and the initial precedence constraint set $\Gamma_v = E$ and $\Gamma_v^T = E^T$. We set $c(0) = 0$, since this node has not been processed. In Step 1, $c(v) = 0$ means that this node is visited for the first time. If $c(v) = 1$ (respectively, 2), then this is the second (resp., third) time to visit v , and we redefine v as the second (resp., third) immediate successor saved in $s(v)$. Finally, if $c(v) = 3$, then we have visited this node three times. If $v > 0$, then we go to the immediate parent node; otherwise, we stop.

In Step 2, we process node v . We first update Γ , $\mathbf{A}_v$ and $\mathbf{A}'_v$ from Γ_v^T , due to adding new precedence relationships as we branch further. We calculate $\underline{ES}_i$ from (PS_i) and $\underline{EF}_i$ from (PC_i) for each task i . Next, we use Algorithm DI to calculate the upper bound UB_v with the learning source sets $\mathbf{A}_v$, and update UB^* if we find a smaller one. We obtain ES_i , the earliest starting time, and LF_i , the latest finish time, for $i \in V$. If the problem has no feasible solution, we do not branch further. Finally, based on the definition of p_i in this new problem, we have $S_i^* \geq ES_i$. Further, since $C_n^*(P) \leq CN$, which is the upper bound on the project makespan, and we use CN as the deadline in the calculation of LF_i , we have $C_i^* \leq LF_i$. Thus, we let $\underline{S}_i = \max\{ES_i, \underline{ES}_i\}$

and $\overline{C}_i = \min\{LF_i, \overline{EF}_i\}$. Then, we compute the lower bound LB_v by solving problem (PR2). If $LB_v \geq UB^*$, then the branch cannot contain an optimal solution.

In Step 3, we process the immediate successors of v . If $\Gamma = \emptyset$, then there are no further branches left, and we go to Step 1. Otherwise, we branch into three cases with partial schedule information: (i^*, j^*) , (j^*, i^*) and $(i^* \parallel j^*)$. Let $s(v) = \{u_1, u_2, u_3\}$ to save the three immediate successors of v , and $\Gamma_{u_1} = \Gamma_v \cup \{(i^*, j^*)\}$, $\Gamma_{u_2} = \Gamma_v \cup \{(j^*, i^*)\}$ and $\Gamma_{u_3} = \Gamma_v \cup \{(i^* \parallel j^*)\}$. We then determine if each of u_1 , u_2 , and u_3 can be fathomed. For example, in the project network in Figure EC.2, if $\Gamma_v = \{(1, 2), (1, 3), (2, 4), (3, 4), (3, 6), (4, 5), (5, 7), (6, 7), (2 \parallel 6)\}$, then $\Gamma_{u_1} = \Gamma_v \cup \{(4, 6)\}$ is infeasible, since $(2, 4)$ and $(4, 6)$ implies $(2, 6)$ which contradicts $(2 \parallel 6)$. Thus, we let $l(u_1) = -1$, which indicates that the branch need not be explored further. Finally, using Lemmas EC.7 and EC.8, we eliminate further branches. A computational study of our branch and bound algorithm appears in Section 7.

Example of Data Modeling and the Formulation for the Case Study. For example, consider task 4, the collection and cleaning of sales data. This task is originally estimated at $t_4 = 20$ days, its nominal processing time. However, information that is learned from a review of the similar task 3 can improve this time. Based on detailed discussions with company officials, we have $\underline{p}_4 = 3$, hence $\alpha_4 = 1 - \underline{p}_4/t_4 = 1 - 3/20 = 0.85$. Further, $q_{34} = 0.5$, which means that tasks 3 and 4 have high learning potential. Then, we have

$$t_4(0.85e^{-0.5by_3} + 0.15) \approx \begin{cases} 20 - 8.5by_3, & 0 \leq y_3 < \frac{1.2}{b}, \\ 20 - 8.5 \cdot 1.2 - 1.7(by_3 - 1.2), & \frac{1.2}{b} \leq y_3 < \frac{5.2}{b}, \\ 3, & y_3 \geq \frac{5.2}{b}, \end{cases} \quad \begin{array}{l} \text{(EC.12a)} \\ \text{(EC.12b)} \\ \text{(EC.12c)} \end{array}$$

and $\bar{y}_3 = \max\{\frac{2.6}{0.5b}, \frac{2.6}{0.5b}, \frac{2.6}{0.35b}\} = \frac{7.4}{b}$ defines the maximum learning time of task 3. Thus, Equations

$$(12)-(14) \text{ for tasks 3 and 4 become: } \quad C_3 - S_3 = t_3 + y_3, \quad \text{(EC.13)}$$

$$C_4 - S_4 = t_4 - (8.5bk_{14} + 1.7bk_{24}), \quad \text{(EC.14)}$$

$$k_{14} + k_{24} \leq y_3, \quad \text{(EC.15)}$$

$$0 \leq k_{14} \leq \frac{1.2}{b}, \quad \text{(EC.16)}$$

$$0 \leq k_{24} \leq \frac{5.2}{b} - \frac{1.2}{b} = \frac{4}{b}. \quad \text{(EC.17)}$$

The problem solved in the Huawei Corporation project is a special case of the general model (1) – (8). In particular, (a) resource constraints are never a bottleneck due to the abundant availability of student workers, and hence they are not modeled, and (b) because of a lack of repetitiveness in the tasks, autonomous learning is very limited and it is therefore not modeled. For these reasons, we now provide a formulation that is simpler and more concise, but nonetheless includes the key features that influence the efficiency of the Huawei project.

$$\begin{aligned} G(\mathbf{A}) &= \min_{\mathbf{S}, \mathbf{Y}, T} T \\ \text{s.t. Constraints (4), (7), (EC.13) – (EC.17) for tasks 3 – 6, 7, 8, 10 – 19.} \end{aligned}$$

Reference

Garey, M.R., D.S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, New York.